

Survey and Taxonomy of Volunteer Computing

TESSEMA M. MENGISTU*, Southern Illinois University Carbondale, USA

DUNREN CHE, Southern Illinois University Carbondale, USA

Volunteer Computing is a kind of distributed computing that harnesses the aggregated spare computing resources of volunteer devices. It provides a cheaper and greener alternative computing infrastructure that can complement the dedicated, centralized, and expensive data centres. The aggregated idle computing resources of devices ranging from desktop computers to routers and smart TVs are being utilized to provide the much needed computing infrastructure for compute intensive tasks such as scientific simulations and big data analysis. However, the use of Volunteer Computing is still dominated by scientific applications and only a very small fraction of the potential volunteer nodes are participating. This paper provides a comprehensive survey of Volunteer Computing, covering key technical and operational issues such as security, task distribution, resource management, and incentive models. The paper also presents a taxonomy of Volunteer Computing systems, together with discussions of the characteristics of specific systems in each category. In order to harness the full potentials of Volunteer Computing and make it a reliable alternative computing infrastructure for general applications, we need to improve the existing techniques and devise new mechanisms. Thus, this paper also sheds light on important issues regarding the future research and development of Volunteer Computing systems with the aim of making them a viable alternative computing infrastructure.

CCS Concepts: • **Computer systems organization** → **Cloud computing**; **Grid computing**; *Client-server architectures*; *Peer-to-peer architectures*; *Reliability*; • **Security and privacy** → *Distributed systems security*; • **Computing methodologies** → *MapReduce algorithms*; *MapReduce algorithms*; • **Software and its engineering** → *Cloud computing*; *Grid computing*;

Additional Key Words and Phrases: Volunteer Computing, Volunteer Cloud Computing, Desktop Grid, Mobile Volunteer Computing, P2P Volunteer Computing, Edge/Fog Computing.

ACM Reference Format:

Tessema M. Mengistu and Dunren Che. 2019. Survey and Taxonomy of Volunteer Computing. *ACM Comput. Surv.* 0, 0, Article 0 (2019), 35 pages. <https://doi.org/10.1145/3320073>

1 INTRODUCTION

The need for more computational resources is on the rise, despite the exponential increase of the computing capacity of workstations [Brock 2006], the proliferation of mobile devices, and the omnipresence of data centres with massive server farms that housed tens (if not hundreds) of thousands of powerful servers. Applications such as biologically inspired algorithms for optimization, huge scientific simulations, business intelligence and big data analysis, etc., need huge computing resources. Supercomputing sites, HPC (High Performance Computing) infrastructures, and large

*This is the corresponding author

Authors' addresses: Tessema M. Mengistu, Southern Illinois University Carbondale, Department of Computer Science, Carbondale, IL, , USA, tessema.mengistu@siu.edu; Dunren Che, Southern Illinois University Carbondale, Department of Computer Science, Carbondale, IL, USA, dche@cs.siu.edu.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2019 Association for Computing Machinery.

0360-0300/2019/0-ART0 \$15.00

<https://doi.org/10.1145/3320073>

Cloud Computing data centres have been used to cater the much needed computing resources. However, the cost of setting up and administering these infrastructures is prohibitively expensive. Moreover, the high energy consumption in data centres contributes to the environmental issue of global warming. Global data centres used roughly 416 terawatts (4.16×10^{14} watts) (or about 3% of the total electricity) in 2016 and this consumption will double every four years [Danilak 2017]. According to [Jones 2018], data centres contribute around 0.3% to overall carbon emissions, whereas the information and communications technology (ICT) ecosystem as a whole accounts for more than 2% of global emissions. The data centres' contribution to the total ICT greenhouse gases emission is expected to increase to 18% by 2020 [Group 2008]. The servers and the cooling system make up about 80% of all the electricity energy used within a data centre [Brown et al. 2008].

Besides the vast number of powerful servers used in these computing infrastructures, there are more than 2 billion underutilized desktop machines that belong to individuals and institutions worldwide [Gartner 2008]. A research showed that Personal Computers (PCs) in institutions/organizations can be idle up to 97% of the time [Domingues et al. 2005]. The current computing landscape is also comprised of more than 50 billion portable devices (mobile phones, laptops, gaming consoles, etc.) connected to the Internet [Evans 2011]. Some of these portable devices have very powerful computing capabilities. Using the spare computing resources of these numerous, underutilized, and individually owned devices can be a cheaper alternative to the expensive data centre based computing infrastructures.

Volunteer Computing (VC) is a kind of distributed computing, where anybody with a computer can donate idle computing resources to run computational and storage intensive tasks [Nouman Durrani and Shamsi 2014]. It is a part of an on-going effort to find a cheaper alternative computing infrastructure for highly resource intensive applications. Since its inception in the late-1980's, Volunteer Computing (a.k.a. cycle stealing system or public-resource computing) has been successfully applied in many compute-intensive projects to solve mainly embarrassingly parallel problems. It has the advantages of providing an alternative relatively inexpensive computing infrastructure for applications and boosting the utilization of the computing resources that would otherwise be wasted. Unlike data centres, VC systems do not require specialized infrastructure, such as battery backups and redundant power systems that accounts 25% of amortized data centre costs [Williams 2004]. Moreover, since VC does not need a dedicated cooling infrastructure that consumes huge electric power, it is a greener solution that can reduce the carbon footprint of the Information and Communication Technology industry.

Volunteer Computing systems are based on donated spare computing resources of volunteers. This makes heterogeneity and intermittent availability of the computing resources as well as best effort service delivery idiosyncratic to VC systems. Applications of VC include, but not limited to, solving scientific problems [Allen 1999; Larson et al. 2002; Sintes 2006; Stainforth et al. 2002], astronomy [Allen 1999; Anderson et al. 2002], smart living [Conner 2010], emergency handling [Nguyen et al. 2017], etc. A typical example of a successful VC project is "The Great Internet Mersenne Prime Search" (GIMPS)¹ that was started in 1996 with the aim of finding the largest prime number. To date, the largest prime number found and verified has 24,862,048 digits. As of January 2018, the project has engaged 182,795 volunteers who provided 1,614,096 CPUs, harnessing a processing power of 346 TFLOP/s.

Different functionalities of a Volunteer Computing system are provided by a middleware and its tasks include [Nouman Durrani and Shamsi 2014]: the partitioning of volunteer tasks into subtasks so that they can run on volunteer nodes efficiently; the optimal scheduling of tasks and resources

¹<https://www.mersenne.org/>

to enhance the performance and improve the robustness of the overall system; modelling the communication among vast number of intermittently available nodes; providing trust and security services, etc. Moreover, VC middlewares support a mechanism to attract and retain volunteers.

This survey paper discusses the technical and operational issues of Volunteer Computing together with solutions and approaches implemented by different VC systems. A detailed taxonomy of the classification of Volunteer Computing systems based on the computational environments that they are deployed in as well as the technical details of typical representative systems in each class are discussed. The paper also sheds light on important issues regarding the future research and development of Volunteer Computing systems with the aim of making them a viable alternative computing infrastructures. Existing survey papers on Volunteer Computing focused on one particular type of VC, such as mobile VC [Tapparello et al. 2015], desktop grid [Nouman Durrani and Shamsi 2014], or a specific VC issue such as task distribution [Choi et al. 2006b]. To the best of our knowledge, this is the first comprehensive survey on Volunteer Computing that covers different kinds of VC systems in Grid, Cloud, P2P, etc., as well as the operational and technical issues pertinent to the successful implementation and deployment of VC systems. The paper also outlines the future directions of Volunteer Computing based on the current research trends in the area.

The rest of the paper is organized as follows. Section 2 reviews the technical and operational issues in VC and discusses a survey of solutions adopted by different VC systems. Section 3 provides the taxonomy of classifications of VC systems. Section 4 outlines the future of Volunteer Computing and Section 5 concludes the paper.

2 OPERATIONAL AND TECHNICAL ISSUES IN VOLUNTEER COMPUTING

A typical VC system is comprised of volunteer nodes that donate their spare resources (*Resource Nodes*), an entity that manages the donated resources and gives a point of entry to volunteers (*Resource Controller*), and users of the VC system. The user submits the task(s) to the *Resource Controller*. After the necessary preprocessing, the *Resource Controller* selects a suitable or a collection of suitable *Resource Node(s)* to deploy the task(s). The *Resource Node(s)* processes the task(s) and returns the result(s) back to the *Controller*. The non-dedicated nature and heterogeneity of the resource base (*Resource Nodes*) of VC systems pose myriads of technical and operational challenges. The following are some of these technical and operational challenges pertinent to Volunteer Computing.

2.1 Architecture

The way the different components of a VC system communicate with each other has a significant impact on the scalability, performance, and security of the system. Based on the deployment of different components <https://boincstats.com/en/stats/-1/project/detail/hostnts>, the architecture of VC systems can be classified as *centralized (client/server)*, *decentralized (P2P)*, and *hybrid*.

In a client/server architecture, a dedicated (collection of dedicated) machine(s) acts as a server and provides resource controller services. The resource nodes will be clients that provide the volunteer computing resources. The central system handles distribution, monitoring, and coordination of tasks together with security related issues. VC systems discussed in [Anderson 2004; Chien et al. 2003; Fedak et al. 2001] follow a typical client/server architecture with asymmetric relationship between the clients and the server. Such centralized architecture creates a bottleneck for scalability and suffers a single point of failure problem. Moreover, the need for a constant network connectivity to the server is another disadvantage of centralized architecture. Generally, in a centralized architecture there is no or limited communication between the volunteer hosts. One of the advantages of centralized architecture is that it is relatively easier for the resource nodes to trust the

system and easier to enforce the security of the whole system at the server side as it has a single point of administration.

In a decentralized (P2P) architecture, a volunteer node can be both a resource node and a controller at same time. In such architecture, communication among nodes will be coordinated without a central authority. Flexibility and scalability are the major advantages of P2P architectures, however, lack of trust is a major hurdle. VC systems discussed in [Andrade and Cirne 2005; Schulz et al. 2008; Shudo et al. 2005; Zhou and Lo 2004] follow a distributed architecture that allows volunteers to donate and consume resources to/from other volunteers directly.

A hybrid architecture blends the flexibility and scalability of decentralized architecture with the security and trust leverage of centralized architecture. In this architecture, the volunteer hosts can communicate with each other and handle some functionalities without intervention from a central server. However, a centralized server still gives functionalities such as global resource directory. Hybrid architecture is deployed in VC systems reported in [Costa et al. 2012; Noor et al. 2016; Soelistio 2013].

2.2 Resource Management

Resource management is one of the crucial tasks in VC systems due to the intermittent availability and heterogeneity of the volunteer nodes as well as the must-to-have non-intrusive characteristics of the systems. This section elaborates on resource management issues pertinent to Volunteer Computing such as discovery and monitoring, non-intrusiveness, intermittent availability, and heterogeneity of volunteer nodes as well as scalability.

2.2.1 Resource Monitoring and Discovery . The non-dedicated nature of volunteer nodes entails high churn rate of computing resources in VC systems. This dynamicity of the availability of computing resources requires continuous monitoring of the volunteer nodes. The monitoring, which has a direct impact on job scheduling decisions, includes the availability of the host as well as the availability of the necessary computing resources to host a guest task/data. Based on the time that the status of a resource is collected, we can categorize resource monitoring as *periodic* monitoring and *on-demand* monitoring [Krauter et al. 2002]. *Periodic* resource monitoring can be *pull-based*, where the resource controller periodically communicates with the individual nodes for state information, or *push-based*, where a resource node regularly sends its state information to the controller [Cao et al. 2004]. The pull method is implemented in VC systems discussed in [Costa et al. 2011; Litzkow et al. 1988; Zhou and Lo 2004] and the push method is implemented in [Vazhkudai et al. 2005]. A combination of pull and push is deployed in some VC systems [Chien et al. 2003; Mengistu et al. 2017, 2018a]. In *on-demand* monitoring, the resource controller pulls the resource nodes whenever it needs resource's state information.

As VC systems do not have a dedicated infrastructure, resource discovery is purely based on membership of volunteers. Membership can be completely voluntary as it is in [Anderson 2004] or depends on organization policy as it is in [Litzkow et al. 1988].

2.2.2 Intermittent Availability . In distributed systems, failure is not an exception rather a norm. A fault tolerant system can perform its functionalities correctly even in the presence of failures. Fault tolerance is one of the crucial characteristics of distributed systems. Volunteer Computing systems depend on donated resources of volunteer hosts, i.e., they do not have a dedicated infrastructure. Volunteers disconnect/connect their resources at any time, thus interrupt the execution of the application or even crash the whole system. This creates intermittent availability of computing resources and it has a significant impact on the reliability and fault tolerance of VC systems, the make-span of the tasks deployed, and the Quality of Experience (QoE) of the VC clients/users.

There are two categories of techniques that mitigate the negative consequences of volunteer nodes' intermittent availability in VC systems: *proactive* and *reactive* techniques. Checkpointing [Ni et al. 2006] and replication/redundancy [Choi et al. 2006a] are reactive techniques. VC systems discussed in [Litzkow et al. 1988; Weissman et al. 2011] use checkpointing and [Anderson et al. 2002; Chandra and Weissman 2009] use replication for reliability. A combination of both checkpointing and replication can also be deployed for fault tolerance [Costa et al. 2012; Fedak et al. 2001]. Prediction techniques that predict the reliability and availability of nodes for future time with some accuracy and take measures to avoid the failure(s) are proactive techniques. Due to the computing resource constraint nature of VC systems, proactive techniques are more appealing than reactive techniques that usually require extra computing resources. Moreover, checkpointing has synchronization overhead that may create a problem for large VC systems.

Prediction techniques for nodes involved in cycle stealing systems have been investigated in the context of desktop grid systems [Lili et al. 2009; Ren et al. 2007], volunteer cloud systems [Graffi et al. 2010], and P2P volunteer computing systems [Ramachandran et al. 2010, 2012]. Prediction techniques can be statistic-based based [Rood and Lewis 2008; Zhang and Phillips 2011; Zhoujun et al. 2007], rule based [Singh and Kaur. 2013], or machine learning based [Nadeem et al. 2008a,b; Ramachandran et al. 2012]. Stochastic modelling techniques, such as a multi-state discrete semi-Markov modelling based prediction algorithm, are also used for the prediction of resource availability in VC systems [Lili et al. 2009; Mengistu et al. 2018b; Ren et al. 2007].

Fitting (un)availability traces of hosts in a grid infrastructure to a statistical distribution, such as Gamma or Weibull, is used to predict the (un)availability of computing resources in [Javadi et al. 2011; Nurmi et al. 2005]. The calculated parameters of the fitted model are used to estimate the future availability of a resource. Such static availability techniques do not take the volunteer hosts' dynamic characteristics, such as a change in resource usage behaviour of users, into consideration for the prediction.

Machine learning based prediction techniques that utilize knowledge of volunteer node's previous and current availability behaviour, collected during the training stage, to predict its future availability are used to remedy the problem of static availability models [Nadeem et al. 2008a]. Unsupervised machine learning that performs clustering analysis on historical data of resource's use of the volunteer node to discover prototypical patterns of use for availability prediction is studied in [Ramachandran et al. 2012]. Hierarchical clustering algorithms, which construct clusters on bottom-up fashion, are used during the training phase. For prediction, the distance between recent history, represented by a vector, and each of the prototypical element of the use classes learned during the training phase is used. Kondo et al. explored the applicability of K-mean clustering for detecting availability patterns of hosts in a volunteer grid [Kondo et al. 2008].

Ren et al. deployed a multi-state semi-Markov modelling based prediction algorithm to estimate the future availability of a node in fine-grained distributed systems [Ren et al. 2007]. Empirical based thresholds for CPU usage and memory trashing are used to identify the state of a compute resource. Discrete Markov model is used to predict the temporal availability of a host for a certain time duration. The authors claimed that average prediction accuracy greater than 86% is achieved. T. Mengistu et al. applied a similar technique for the prediction of availability and reliability of nodes in volunteer cloud systems [Mengistu et al. 2018b]. M. Rood and J. Lewis proposed a multi-state prediction model that used condor based state classification [Rood and Lewis 2008]. Algorithms based on a volunteer host's past N days of availability behaviour (N-Day) versus most recent N hours of activity (N-Recent) are deployed. Two trivial statistical analysis, the proportion of time nodes stayed in each state and the state transition counting, combined with weighting scheme are used for future availability prediction of machines.

Ramachandran et al. investigated the effectiveness of prediction for scheduling tasks in a P2P VC environment [Ramachandran et al. 2010, 2012]. Individual machine's availability is denoted by resource availability factor (*raf*), which is the probability that a machine is available at a specific time. From their experimentation, in a simplified environment, the authors claimed that the number of migrations during the job execution is decreased due to the prediction.

2.2.3 Non-intrusiveness. One of the disadvantages of Volunteer Computing is the resource stress that guest applications may impose on the volunteer nodes, if their usage is not controlled and regulated. The controlling and regulation of the resource usage of a guest application in such a way that it does not affect the primary purpose of the volunteer node is called non-intrusiveness (a.k.a. unobtrusiveness) of the VC system. Non-intrusiveness is one of the important design goals of VC systems not only to attract new volunteers but also to retain existing ones. Continuous monitoring of the resource utilization of guest applications is important to guarantee the non-intrusiveness of VC systems. Different mechanisms are deployed in different VC system in order to guarantee non-intrusiveness. Some VC systems activate guest applications only as screen saver when resources are completely idling [Anderson 2004; Anderson et al. 2002; Andrade et al. 2003] or when there is no application running and there is no keyboard or mouse movement [Litzkow et al. 1988]. T. Mengistu et al. explored enforcing non-intrusiveness based on the resource availability thresholds [Mengistu et al. 2017]. In their system, if the resource usage (such as CPU and RAM) of the local processes exceeds a predefined threshold, the guest process will be migrated to another available volunteer node or will be killed and restarted. Running guest tasks at lowest process and thread priority is also implemented for unobtrusiveness in [Chien et al. 2003].

2.2.4 Heterogeneity. The heterogeneity in VC systems lies in both the hardware and software of the volunteer nodes. A volunteer node can be a desktop, laptop, cellphone, game console, networking device, or smart TV. The hardware of volunteer nodes can be extremely diverse in terms of processor speed and type, RAM, disk space, I/O capacity, and networking capability in terms of services and communication technologies (bandwidth, WiFi, Bluetooth, etc.). In addition, there are diverse range of operating systems and versions on volunteer devices. In general, volunteer device's attributes are categorized as *static* or *random* [Nouman Durrani and Shamsi 2014]. Attributes that do not change in a short period of time are *static* attributes and those attributes that show random behaviour within a short period of time are *random* attributes. CPU frequency, RAM capacity, Hard disk size, etc., are static attributes whereas free RAM capacity, free Hard Disk storage space, battery level of cellphones, etc., are example of random attributes. The heterogeneity and diversity of volunteer nodes create a challenge for optimal distribution of tasks to volunteer nodes according to their capability [Schmohl and Baumgarten 2008]. Different task distribution algorithms, such as weighted-round-robin, buffer-n, first-come-first-served, etc., have already been practised in VC with the goal of decreasing the overall makespan of applications that runs over different volunteer nodes with different capability [Anderson 2011; Toth 2008; Toth and Finkel 2008, 2009]. Running guest applications in virtual machines or sandbox is another solution for the heterogeneity problem. Targeting specific devices, such as PCs only or cellphones only, as volunteers is a simple approach to handling device heterogeneity in VC.

2.2.5 Scalability. VC systems can have millions of volunteer nodes connected to them. For example, BOINC² has more than 1.1 million volunteer computers currently. A novel and scalable approach is required to handle these big number of volunteer nodes coupled with their intermittent availability as discussed above. Using a decentralized architecture and distributing different

²<https://boincstats.com/en/stats/-1/project/detail/host>

components/functionalities of the VC system in a P2P fashion are deployed to support scalability [Adya et al. 2002; Butt et al. 2004]. Having a distributed server architecture that can run over multiple machines is another solution for scalability in VC systems [Anderson 2004].

2.3 Task Distribution/Scheduling

Efficient task distribution and optimal task scheduling are crucial for the success of VC systems. Most of the existing task distribution policies in VC systems can be classified as: *naive scheduling* policies and *knowledge-based* policies [Nouman Durrani and Shamsi 2014].

Naive scheduling policies distribute tasks irrespective of the history of the volunteer nodes. Following are some naive scheduling algorithms. *First-Come-First-Served (FCFS)* scheduling policy assigns tasks to any volunteer node that requests a task without consulting the availability and reliability history of the node [Estrada et al. 2006, 2009; Taufer et al. 2007]. Due to this blind distribution policy of FCFS, there is a chance that tasks will be distributed to nodes with availability and reliability issues in the past. This may result in the degradation of performance of the VC system as a whole [Anderson et al. 2005; Estrada et al. 2008]. In *Random assignment* policies, tasks are assigned to randomly selected volunteer nodes [Anderson 2011; Kondo et al. 2007]. To enhance the performance of systems, random assignment policies can assign tasks randomly to a group of volunteer nodes with similar patterns. *Locality based* scheduling policy assigns tasks to those volunteer nodes that possess local data to handle the task [Anderson et al. 2005; Estrada et al. 2008; Ryden et al. 2014].

Knowledge-based policies take the history of the volunteer nodes, requesting for tasks/data, into consideration to assign the tasks/data. *World Community Grid* distributes work units based on the average turnaround time, which is the mean time a volunteer node requires to process a task under a deadline constraint [Estrada et al. 2008; Toth 2008; Toth and Finkel 2009]. The *Weighted-Round-Robin* policy time slicing among projects of different servers weighted, for example, based on their deadline [Anderson 2011; Anderson and McLeod 2007; Kondo et al. 2007]. In the *Work-Fetch* policy, the volunteer nodes are allowed to keep enough tasks queued for a certain duration, for example network round-trip-time. Reliability and availability thresholds are also used for task scheduling. In such policies, a fixed or variable availability and reliability thresholds will be defined. Tasks will not be assigned to a volunteer node that does not meet the threshold criteria [Estrada et al. 2006, 2008, 2009]. The policy has a problem of starvation, where workers may be forced to wait for tasks to be assigned till they meet the threshold criteria. S. H. Ngo et al. proposed a priority queue based round robin policy, where volunteer nodes are enqueued based on the amount of time (follows Gaussian distribution) they need to execute a task [Ngo et al. 2008]. Tasks are assigned to volunteer node according to their dequeuing order. Another policy that is used in *grid.org*³ and *Xtremweb* [Fedak et al. 2001] is the *buffer-none* policy, where a volunteer host downloads, processes, and returns a result one task at a time before it asks for another task from the project's server. The major drawback of this policy is waste of CPU cycle while downloading the new task, specially if there is a need to download a large data file [Toth and Finkel 2008, 2009]. The *Buffer-N* policy, on the other hand, downloads and buffers multiple tasks for N days or hours and processes them one at a time. A node is allowed to download another chunk of tasks, when it finishes processing all previously downloaded tasks [Toth and Finkel 2008, 2009].

Genetic algorithms are also deployed for task distribution in VC systems [Desell et al. 2010; Estrada et al. 2008]. X. Want et al. discussed a look-ahead genetic algorithm (LAGA) that employs time dependent reputation, called reliability-driven reputation (RD), for workflow applications [Wang et al. 2011]. S. H. Ngo et al. proposed a Poisson process based communication model of task

³<http://www.grid.org>

Table 1. Summary of Task Distribution/Scheduling Techniques in Volunteer Computing Systems

	Algorithm	Example VC Systems
Naive	FCFS	[Estrada et al. 2009][Taufer et al. 2007][Estrada et al. 2006]
	Random Assignment	[Kondo et al. 2007][Anderson 2011]
	Locality Based	[Anderson et al. 2005][Ryden et al. 2014] [Chandra and Weissman 2009] [Ryden et al. 2014] [Hamdaqa et al. 2015]
Knowledge-Based	Average Turnaround Time	[Toth and Finkel 2009][Toth 2008]
	Round Robin	[Kondo et al. 2007][Anderson 2011][Anderson and McLeod 2007][Ngo et al. 2008] [Litzkow et al. 1988]
	Buffer-N	[Toth and Finkel 2008] [Toth and Finkel 2009][Fedak et al. 2001]
	Work-Fetch	[Anderson and McLeod 2007][Kondo et al. 2007]
	Reliability/Availability Threshold	[Sonnek et al. 2007][Mengistu et al. 2017][Estrada et al. 2009][Estrada et al. 2008][Estrada et al. 2006][Hamdaqa et al. 2015]
	Genetic Algorithm Based	[Wang et al. 2011] [Desell et al. 2010][Estrada et al. 2008]
	Probabilistic Based	[Ngo et al. 2008]
	Trust/Reputation Based	[Kuada and Olesen 2011] [Chard et al. 2012b]
	Heuristic Based	[Panadero et al. 2017] [Ghafarian et al. 2013] [Ghafarian and Javadi 2015][Guler et al. 2015]

requests for volunteer nodes where the request rate is controlled through the nodes reconnection period (T) [Ngo et al. 2008]. Reputation/trust based task distribution is proposed in [Chard et al. 2012b; Kuada and Olesen 2011]. J. Sonnek et al. [Sonnek et al. 2007] discussed an adaptive scheduling technique that employs a dynamic reputation based reliability of a node for efficient task distribution and redundancy together with voting based verification. The reliability of a worker is the ratio of successful completion of assigned tasks. Heuristics based algorithms are also used for task scheduling [Ghafarian et al. 2013; Ghafarian and Javadi 2015; Guler et al. 2015; Panadero et al. 2017]. For example, H. Guler et al. presented many heuristics that distribute tasks with the objectives of maximizing the throughput of the workers while taking the budget constraints into consideration [Guler et al. 2015]. Ghafarian and Javadi developed algorithms to schedule workflows of scientific and data intensive applications for meeting the deadline and satisfying Quality of Service (QoS) constraints [Ghafarian and Javadi 2015]. Table 1 gives the summary of the task distribution/scheduling techniques in VC systems.

2.4 Business/Incentive Model

Currently, the participation of volunteers in VC systems is predominantly based on personal interests in some projects or because of belongingness to a certain community. In order to attract volunteers to the VC arena, it is vital to have a business model that includes economic incentives

to encourage the volunteers not only to donate resources but also continue donating. Different business/incentive models that have been proposed in VC systems are discussed in the following paragraphs.

Fixed price in the form of token exchange together with peer approved service quality reputation based rating for file sharing is investigated in [Ranganathan et al. 2004]. N. Andrade and W. Crine proposed a reputation based technique that rewards nodes that donate more resources [Andrade and Cirne 2005]. An incentive mechanism that uses the percentage of resources a volunteer node contributes in a P2P VC system is discussed in [Rahman et al. 2010; Vega et al. 2013]. In this effort-based approach, the node's resource contribution, compared to its overall resource capacity, is considered. The proposed effort-based approach tries to be fair by benefiting nodes with small resource capacity and it also handles requests from nodes with higher resource capacity. Another approach that uses concept of "local communities" identified as a form of social graph is proposed in [Mohaisen et al. 2011]. In this model, every node generates its own currency whose value changes depending on the node's reputation in the market. The authors assumed members of a social network belong to a community where participants are connected directly or a few hops away. Using a virtual currency, the trust inherent within social networks, and decentralized reputation management system, the author showed how social network density and size, together with clustering strategy and reputation distributions affects the transactions' success and the credit gain using simulation. Table 2 summarizes the business/incentive models in VC systems. In general, business/incentive models in VC can be classified as follows [Chard et al. 2012b]:

- *Volunteer*: Volunteers contribute resources without expecting monetary or non-tangible gain [Anderson 2004; Harvey et al. 2007].
- *Trophy*: Volunteers are awarded with non-tangible (non-monetary) credits or prizes for their quantifiable contributions [Cooper et al. 2007].
- *Reciprocation*: A rewarding scheme where users with higher contribution to the VC system are given priorities when they need resources [Andrade et al. 2009; Andrade and Cirne 2005; Andrade et al. 2003; Park and van der Schaar 2010].
- *Reputation*: A reputation that is based on the interactions in the community is used to allocate resources [Hendriks et al. 2015; Jøsang et al. 2007; Sabater and Sierra 2002; Wu and Wu 2009].
- *Posted price*: In this model, resources are provided at a set (posted) market price [Li et al. 2007; Seifert 2006].
- *Auction(tender)*: A market price for a resource is set by an auction among multiple contenders [Buyya and Venugopal 2004; Lin et al. 2010a; Waldspurger et al. 1992].

The number of participating volunteer nodes in VC systems has a big significance on their performance. Thus, retaining already existing volunteers is as important as the need to attract new volunteers. Based on a qualitative study of volunteers in the climate prediction.net project, Darch and Carusi proposed mechanisms to retain volunteers by providing a scoring system for volunteers that shows their contribution accurately [Darch and Carusi 2010]. Moreover, information regarding the outputs of the VC project should be readily accessible to build the volunteers' confidence on the worthiness and productivity of the project that they are involved in [Tapparello et al. 2015]. Informing the volunteers about the purpose and result of the project and making them aware about cost/performance when using VC is also proposed as retention approach [Dorneanu 2011]. Gamification, which is the use of principles of game design in a non-game context [Deterting et al. 2011], is another technique proposed to retain volunteer nodes. In their position paper, A. Shahri et al. proposed the use of gamification to motivate volunteers to participate and continue

Table 2. Summary of Business/Incentive models in Volunteer Computing Systems

	Description	Example VC Systems
Volunteer	Volunteers donate resources without expecting a reward.	[Anderson et al. 2002][Anderson 2004][Harvey et al. 2007]
Trophy	Volunteers are awarded with non-monetary (non-tangible) credits or prizes.	[Anderson et al. 2002][Cooper et al. 2007][Chard et al. 2012a][Shahri et al. 2014]
Reciprocation	Volunteers with higher contribution to the VC system are given priorities when they need resources.	[Andrade et al. 2009][Andrade et al. 2003][Park and van der Schaar 2010][Andrade and Cirne 2005][Vega et al. 2013][Rahman et al. 2010]
Auction(tender)	Market price is set for a particular resource.	[Waldspurger et al. 1992][Buyya and Venugopal 2004][Lin et al. 2010a][Mohaisen et al. 2011]
Reputation	Based on a measure of individual reputation.	[Hendrikx et al. 2015][Jøsang et al. 2007][Sabater and Sierra 2002][Wu and Wu 2009][Andrade and Cirne 2005][Noor et al. 2014]
Posted price	Resources are provided at a set (posted) price.	[Li et al. 2007][Seifert 2006][Ranganathan et al. 2004]

participation in volunteer cloud computing projects [Shahri et al. 2014]. The authors discussed different techniques of game design that can be used in different classification of volunteers.

A free rider problem [Sirivianos et al. 2007], which is a problem of using other volunteers' resource without contributing to the ecosystem, is a problem specifically in P2P Volunteer Computing systems. Reciprocation is deployed to handle free rider problems [Andrade and Cirne 2005; Andrade et al. 2003].

2.5 Security

Security is one of the most researched issues in computing in general and distributed systems in particular. Integrity, confidentiality, and availability are the three factors that affect the security of a computing system. Due to the inherent nature of Volunteer Computing, trust relationship between a VC system and the volunteer nodes should not be assumed. The lack of trust is mainly due to the anonymity of volunteer nodes coupled with their dynamic availability. This makes security an indispensable component that is integrated in various functionalities of VC systems, such as discovery of volunteers, partitioning, distribution, and processing of tasks/data, communicating the results, and even incentive mechanisms. Some of the possible security attacks in VC systems include: result falsification, where the attackers return incorrect results; malicious executables distribution by a VC project; trying to increase battery lifetime by driving other volunteer mobile devices to execute its tasks; accessing information residing in the volunteer host by the guest application; etc. A VC project may release an application with bugs that breaches the security of the participant hosts, for example, tampering with system files within the host or deleting the host's files [Cano and Vargas-Lombardo 2012].

Some of the solutions for result falsification deployed by VC systems include having multiple answers for the same problem using replication and the answer returned by the majority can be

taken as the correct answer [Anderson et al. 2002; Costa et al. 2013]. *M-first voting*, where the task will be executed repeatedly until the first m matching results are collected by the server, is discussed in [Anderson 2004; Watanabe and Fukushi 2010; Watanabe et al. 2011]. A major drawback of these voting techniques is that they waste the scarce donated resources by replicating the tasks. Another technique is *spot-checking*, where the server sends a job whose answer is known beforehand to volunteer nodes. If a volunteer node returns an incorrect result for that spotter job, the host is identified as a malicious node and it will be blacklisted. This will result in all subsequent results from that node be invalidated or no job will be assigned to the node [Andrade and Cirne 2005; LFG 2001; Sarmenta 2001; Watanabe and Fukushi 2010].

Digital signature based data and executables distribution is a solution for the malicious project security problems. For example, each project running under BOINC is signed and the signing key pair changes periodically. Encryption is used to keep the confidentiality and integrity of the tasks/data at rest on the volunteer hosts [Fedak et al. 2001; Mengotti 2004] as well as during transit in a network. Sandboxing and virtualization technologies can be used for deployment of guest tasks so that they will be executed in a controlled environment [Cappello et al. 2005; Cappos et al. 2009; Chien et al. 2003; Zhou and Lo 2004]. This will prevent intentional as well as unintentional security attacks from a project server. Denial of Service (DoS) attacks target VC projects by sending a very large data/result to the project server or very large computational task to a peer volunteer node by a malicious volunteer node. Using upload authentication key pair and limiting the maximum size for each result/data are solutions for DoS security threats. Some VC systems use the trust relationship among friends in social network applications such as Facebook and Twitter to enforce the trust relationship between the VC system and volunteer nodes [Chard et al. 2012b,a; Kuada and Olesen 2011; McMahon and Milenkovic 2010; Mohaisen et al. 2011].

3 TAXONOMY OF VOLUNTEER COMPUTING SYSTEMS

Volunteer Computing, which harnesses the aggregated spare computing resources of volunteer devices, is deployed in different computational environments and contexts. In this survey paper, we classify Volunteer Computing systems based on the computational environments that they are deployed in as: *volunteer grid*, *volunteer cloud*, and *volunteer mobile systems*. The majority of VC systems are geared towards embarrassingly parallel tasks that need no or little communication and distributed/scheduled as BoT (Bag of Tasks). However, recently the need for VC systems that can handle parallel tasks with heavy communication is on the rise. Hence, we discuss those systems as a separate class of VC systems in this paper. Table 3 gives the taxonomy of VC systems discussed in this section. The following subsections discuss the classification in detail.

3.1 Volunteer Grid Computing

Volunteer grids are one way of fulfilling the original goal of Grid Computing, where anyone can donate computing resources to the grid so that it can be used by users for their computational needs. Contrary to the traditional grid infrastructure, which needs a dedicated infrastructure to run on, volunteer grid runs on scavenging computing resources from desktop computers for computationally intensive applications. We can classify volunteer grid systems as [Fedak 2010]: *Public Resource Computing (PRC)* and *Enterprise Desktop Grid Computing (EDGC)*. *PRC* utilizes geographically dispersed, but networked, PCs to solve very large computational tasks, such as scientific researches. *EDGC* utilizes the spare computational capacity of PCs within an organization/enterprise to execute large-scale applications such as weather forecast. User participation in *EDGC* is not always voluntary and is dictated by enterprise policies [Lerida et al. 2013], however, *PRC* systems are purely based on the voluntary participation of the computing machines' owners. The following paragraphs discuss selected volunteer grid systems.

Table 3. Taxonomy of Volunteer Computing Systems

	Classification	VC Systems
Grid	Enterprise Desktop Grid (EDGC)	[Castro et al. 2010][Litzkow et al. 1988][Chien et al. 2003][Schulz et al. 2008] [Mengotti 2004] [Chu et al. 2007]
	Public Resource Computing (PRC)	[Urbah et al. 2009][Anderson et al. 2002] [Herr et al. 2006] [Adam-Bourdarios et al. 2015][Urbah et al. 2009][Fedak et al. 2001][Anderson 2004][Andrade and Cirne 2005][Brasileiro et al. 2007][Zhou and Lo 2004]
	Storage Grid	[Ma et al. 2009][Fedak et al. 2009][Butt et al. 2004][Adya et al. 2002]
Cloud	Desktop Cloud	[Mengistu et al. 2018a][Distefano et al. 2016][McGilvary et al. 2015][Ryden et al. 2014][Osorio et al. 2012][E. Rosales and Villamizar 2011][Kim et al. 2017] [Briscoe and Marinos 2009][Anjomshoa and Salleh 2014][Hari et al. 2012][Cappos et al. 2009] [Ahmed et al. 2013]
	P2P Cloud	[Graffi et al. 2010][Babaoglu et al. 2012]
	Storage Cloud	[Moreno-Martínez et al. 2012][Beberg and Pande 2007][Qin et al. 2015][Neumann et al. 2011][Noor et al. 2016]
	SocialCloud	[Kuada and Olesen 2011][Caton et al. 2014][Chard et al. 2012a][McMahon and Milenkovic 2010][Mohaisen et al. 2011]
	Edge Cloud	[Alonso-Monsalve et al. 2017][Hasan et al. 2015]
Mobile	Server Driven	[Ba et al. 2013][Arslan et al. 2012][Habak et al. 2015][Mtibaa et al. 2013][Miluzzo et al. 2012] [Cuervo et al. 2011]
	User Driven	[Shi et al. 2012]overall [Noor et al. 2014][Cornelius et al. 2008][Gordienko et al. 2015][Das et al. 2010][Agapie et al. 2008][Gaonkar et al. 2008][Deng and Cox 2009][Lu et al. 2010]
Parallel		[Hamdaqa et al. 2015][Bruno and Ferreira 2014][Lin et al. 2010b][Costa et al. 2011][Jin et al. 2012][Costa et al. 2013] [LeBlanc et al. 2010][Marinelli and Tan 2009][Calderón et al. 2012][Costa et al. 2012][Rezgui et al. 2018]

Entropia: is a desktop grid system that aggregates the desktop computer resources of an enterprise into a virtual infrastructure. The system is developed in such a way that it fulfils the requirements efficiency, fault tolerance, security, unobtrusiveness, and manageability as well as easy application integration [Chien et al. 2003]. The main idea behind Entropia, as it is with other desktop grids, is to harness the idle computing resources of desktop computers. Entropia server architecture follows a layered approach with three layers: the *physical node management*, *scheduling*, and *job management* layers. The *physical node management* layer gives services such as resource management, communication, security, etc., in addition to reliability services. A proprietary binary sandboxing used in this layer helps Entropia to be secured and unobtrusive to the host that it is running on [Chien et al. 2003]. The sandboxing also helps the users to deploy their application without the need for modification. The *Resource Scheduling* layer schedules jobs for execution on physical machines, taking the heterogeneous, dynamic, and unreliable nature of the resource pool into consideration. The *job management* layer is the one that handles the submitted jobs including the division of the job into subjobs and making sure that each job executes correctly. Entropia also uses a component called *Desktop Client* that resides on a client machine and makes sure that the jobs are unobtrusive to the host. The unobtrusiveness is achieved by making the Entropia tasks run at a low process and thread priority. *Desktop Client* also constantly monitors the resource utilization of a host and pauses or stops the job, if the resource usage is high to avoid contention. Entropia supports only Windows machines to create the grid infrastructure.

Condor: is a desktop grid system that schedules long running background jobs on idle workstations [Litzkow et al. 1988]. Automatic placement of jobs on idle workstations, reliability, unobtrusiveness, and fairness are the design goals of the condor system. Every workstation has a local scheduler and a queue for background jobs. A job submitted by a user will be placed in a background queue. A centralized coordinator polls every workstation in the system periodically to see if they are available for running background jobs and/or have a background job to be executed. The coordinator allocates capacity on idle machines to local schedulers who have background jobs in their queue. If a background job is scheduled in a workstation, the machine checks every half minutes if the local user resumed using the workstation. If the local user resumes using the machine, the background job will be preempted in order to make the condor system unobtrusive. For reliability, condor uses checkpointing. A priority based job scheduling algorithm is used to fairly distribute the spare cycles among heavy and light users. A metrics called *leverage*, which is the cost of running a job remotely over the cost of running the same job locally, is introduced in condor. Only jobs that have large *leverage* should be scheduled on remote spare cycles. Condor scheduler only schedules batch jobs.

OurGrid: is a P2P community based grid system that provides resources for applications that can run over non-dedicated infrastructures [Andrade and Cirne 2005]. An institution that contributes resources to the resource pool and uses computing resources from the pool when the need arises is considered as a peer. OurGrid has three components: *OurGrid Toolkit*, a component that runs Bag of Task applications on grids; *MyGrid*, provides the user with high-level abstractions (such as tasks, jobs, and grid machines) to deal with the grid; *SWAN*, provides secured resource access services. A reputation based incentive in the form of more resource donation with more reward is implemented in OurGrid. In order to avoid the free-riding problem, OurGrid uses reciprocity. Isolation using sandboxing is used to protect the system against malicious applications. The system is in production since 2004 and has been used by many applications such as hydrological resource management [Brasileiro et al. 2007].

BOINC⁴(Berkeley Open Infrastructure of Network Computing): is a widely used tool that is based on millions of volunteer computers worldwide for highly compute intensive tasks of public interest. It gives both computing and storage resources for users [Anderson 2004]. BOINC is a general-purpose middleware that facilitates an easy access and a cheap provision of computing resources for scientists in the context of Volunteer Computing. The server side of the BOINC system is a collection of web services and daemon processes. It uses relational database for storage. Scheduling servers handle work unit distribution to clients and keep track of the completion of the tasks. The data server handles file uploads using certificate based mechanisms. Volunteers, who want to participate in one or more of the projects that use BOINC, should register and download the *BOINC client*. BOINC uses a credit based incentive mechanism, where volunteers are rewarded based on their contribution towards the handling of jobs. Redundancy is used to handle malicious users and exponential back-off technique is deployed in case of failure for scalability. Currently, BOINC hosts 37 officially supported VC projects⁵ such as IBM World Community Grid, Milkyway@home, Einstein@home, Rosetta@home, SETI@home, etc. As of January 2018, there were 4,546,997 users and 1,095,412 participating computers with total of 21.683 PetaFLOPS donation.

SETI@home: is an Internet distributed computing project and one of the most widely referenced projects in the area of Volunteer Computing. SETI (Search for ExtraTerrestrial Intelligence) is a scientific effort that tries to find life outside of our planet earth. SETI@home allows anybody with a computer and an Internet connection around the globe to participate in the project. Currently, the project has more than 1.7 million registered volunteer users worldwide⁶. The architecture of the system is a client/server, where the server creates and distributes the work units to the clients. The clients download work units, analyse them, and return back the results to the server. The work units of SETI@home are embarrassingly parallel tasks that will be downloaded by clients when they are idle. SETI@home detects and discards results from malicious users, by doing redundant computation for each work unit [Anderson et al. 2002].

XtremWeb: is designed to provide a computing platform to meet large computing demands worldwide [Fedak et al. 2001]. XtremWeb is designed in such a way that a registered collaborator can create its own VC infrastructure that can cooperate with the main XtremWeb server. XtremWeb's architecture follows a three-tier design: *worker*, *client*, and *coordinator*. The *worker* (volunteer node) has a full control on how to share the spare resource. Multiple threads, background thread, control thread, compute thread, and alive thread dictate the behaviour of a node. The task requests submitted by *clients* will be executed on the volunteer nodes. The *coordinator* of XtremWeb has different components: *Dispatcher* has the responsibility of distributing tasks to the scheduler; *Scheduler* is the one that assigns a task to a worker and keep track of the task; Database of tasks/jobs and their related information. Encryption and checksum are used on the top of sandboxing to make XtremWeb secure.

Similar volunteer grid systems include: *CCOF (Cluster Computing On the Fly)*, a P2P cycle stealing system that harnesses spare resources of PCs over the Internet [Zhou and Lo 2004]; *Alchemi*, a .NET-based Grid framework with a hierarchical architecture to support parallel communicationless and scalable applications [Luther et al. 2005]; *GPU (Global Processing Unit)*, a framework that builds a cluster based on a P2P desktop resource aggregation [Mengotti 2004]; *Cohesion*, a P2P desktop grid system that is based on microkernel approach for realizing parallel programming models and applications [Schulz et al. 2008]; *EDGEs (Enabling Desktop Grids for e-Science)*, an European initiative that connects the EGEE Grid infrastructure with desktop grid systems in order

⁴<https://boinc.berkeley.edu/>

⁵<https://boinc.berkeley.edu/projects.php>

⁶<https://boincstats.com/>

to support the specific needs of scientific communities [Urbah et al. 2009]; *unaGrid*, a virtual opportunistic grid infrastructure that provides computing infrastructure for scientific tasks [Castro et al. 2010]; *Aneka*, a desktop grid platform based on .NET framework [Chu et al. 2007]; and the @Home projects such as, ATLAS@home [Adam-Bourdarios et al. 2015], LHC@home [Herr et al. 2006], etc.

The majority of the above volunteer grid systems predominately focus on providing computational services specifically for high throughput applications. However, in addition to the above systems, there are also volunteer storage grid systems.

FreeLoader: is a decentralized system that harnesses the unused storage space and I/O bandwidth of volunteer machines to provide a shared cache space for hosting scientific datasets [Vazhkudai et al. 2005]. It provides a lightweight cache storage space, rather than to be a general-purpose distributed file system. It has two components: the *management component* that handles operations, such as lookup services and replication; the *storage component* that consists of nodes that donate storage resources (also called *benefactors*). A fragments of immutable datasets, called *morsels*, are distributed among the benefactors. Metadata checkpointing is used together with cache replacement and replication for reliability. A prototype based experimentation showed a good performance and a small overhead on volunteer nodes, such as a 1.5% overhead on CPU and a 25% network latency on Web transfers. An enhanced version of FreeLoader [Ma et al. 2009] provides additional functionalities, for example collective file transfer and prefix-caching, which improve applications' performance, with regard to accessing cached data.

Similar volunteer storage grid systems include: *Farsite*, a system that harnesses the aggregated resources of autonomous, insecure, and unreliable volunteer computers to provide a reliable and secured file storage service [Adya et al. 2002]; *Kosha*, a blend of P2P and Network File System (NFS) that provides a reliable and shared file system by using the redundant storage space on cluster nodes and desktop computers [Butt et al. 2004]. *BitDew*, provides a meta-data driven data management operations, for example fault-tolerance using replication, to computational desktop grids [Fedak et al. 2009].

3.2 Volunteer Cloud Computing

Cloud Computing is “a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction” [Mell and Grance 2011]. The conventional centralized data centre based Cloud Computing has the advantage of economy of scale that allows a cheaper provision of computing resources to customers by reducing Total Cost of Ownership (TCO). However, the need for large initial investment to set up the data centres and the day-to-day maintenance cost coupled with the risk of single point of failure, even with redundancy, are major challenges in Cloud Computing. Moreover, the high energy consumption of data centres contributes to the environmental issue of global warming.

The pay-as-you-go business model of clouds makes computing services and resources like commodities that we can purchase. However, a centralized data centre based public Cloud Computing is not suitable for all application scenarios [Mengistu et al. 2017]. Federated clouds and volunteer clouds have been investigated as alternative cloud infrastructures to the centralized data centre based clouds [Varghese and Buyya 2018]. Volunteer clouds are opportunistic cloud systems that runs over the donated spare resources of volunteer computers. In this paper we classify volunteer cloud systems as desktop cloud, P2P cloud, social cloud, volunteer storage cloud, mobile cloud, and edge cloud. The following paragraphs discuss these classes of volunteer cloud systems respectively, except edge clouds that will be discussed in Section 4.

3.2.1 Volunteer Desktop Clouds. The development of a cloud computing model based on idle resources of general-purpose computers in offices and homes as its resource base is discussed by Kirby et al. [Kirby et al. 2010] and Che and Hou [Che and Hou 2011]. The authors explored many research issues with regard to developing desktop based cloud computing model. They called their model Ad Hoc Cloud and Credit Union Cloud Model (CUCM) respectively and proposed possible architectures as a starting point. Ad Hoc Cloud is defined in [Kirby et al. 2010] as a union of a set of cloudlets, each of which provides a particular service or application. Each cloudlet runs on intermittently available physical machines that may host multiple cloudlets. A cloud element, which is a software, will be assigned to each cloudlet. The Ad Hoc cloud contains different components; an element manager that creates and destroys cloud elements; a component that monitors the effects of the cloud element on the user process; a component that handles the QoS issues, etc. Credit Union Cloud Model proposed in [Che and Hou 2011] is very much inspired by the business model of real credit unions. The authors advocate to treat spare/underutilized CPU cycles and storage spaces the same way as ones spare money and to deposit/contribute to the resource pool of a special cloud, Credit Union Cloud, where individually owned compute resources get consolidated into a large cloud fabric to back up a community's self-provisioned cloud. Credit Union Cloud Model has the greatest appeal to institutions and organizations such as colleges, universities, and government agencies not only they get free clouds (based on their existing infrastructures) but also they retain full control, one of the biggest obstacles preventing organizations with high confidentiality requirement moving to the public clouds. The following are examples of volunteer desktop cloud systems.

cuCloud: is an opportunistic private/public cloud system that runs over idle computing resources of machines within an organization/institution [Mengistu et al. 2017], directly based on the Credit Union Cloud Model [Che and Hou 2011]. The cuCloud is a Volunteer Computing as a Service (VCaaS) cloud system that executes over an existing computing infrastructure, which is not specifically setup for cloud purpose [Mengistu et al. 2018a]. The cuCloud follows a client/server architecture, where the clients are the volunteer hosts that donate computing resources for running guest Virtual Machines (VMs). Every volunteer node has a piece of software called *Membership Controller (MC)* that controls the membership of the node to the virtual data centre of cuCloud. The MC has sub-components such as the *Sensor* that monitors the resource utilization of the node periodically and gives that information to the *Node Profiler*. The *Node Profiler* calculates the reliability and availability score of the volunteer node based on historical data and the current resource utilization [Mengistu et al. 2018b]. The reliability and availability score will be sent to the server via the *Reporter* component. The server, which is based on CloudStack ⁷, manages the volunteer nodes. It has different components: the *Resource Manager and Allocator* component has the information regarding the computing/storage resources the cuCloud infrastructure possess; *Scheduler* accepts customer's requests and schedules the requests according to the availability of resources in the resource pool; *Virtual Machine Manager* component handles the deployment and monitors the progress of VMs on volunteer hosts; *Security Module* handles the security of the VMs. To enforce non-intrusiveness cuCloud uses type II hypervisors and threshold based resource utilization monitoring. The cuCloud uses resource utilization and reliability aware VM scheduling algorithms to increase overall resource utilization of the system and minimize VM migrations [Mengistu 2018]. After testing cuCloud for CPU and memory bound tasks, the authors concluded that the "No Data Center" approach of Cloud Computing using volunteer nodes is a feasible and economical alternative to the conventional Cloud computing infrastructures.

⁷<https://cloudstack.apache.org/>

AdHoc Cloud: is a volunteer cloud system that runs over a set of non-dedicated, dynamically available, and unpredictable machines, with no assumption about trust relationship between the tasks and the hosts that run them [McGilvary et al. 2015]. The idea of AdHoc Cloud is to change the non-dedicated, unreliable, and idle computing resources from a locally owned infrastructure into a volunteer cloud computing infrastructure. The system follows a client/server architecture. The server has three subcomponents: *Cloud Interface*, *VM Service*, and *Job Service* components. The *Cloud Interface* is where the cloud user submits jobs and the resource donor donates resources. The *Job Service* handles the jobs submitted by clients. The server schedules jobs and VMs to hosts, monitors the status of jobs, and manages the overall system. The scheduler selects the near optimal hosts for a job by first selecting hosts with enough resources and then orders them according to their reliability. Reliability is calculated based on the percentage of tasks handled successfully in the past and current availability of the node. The system is based on BOINC with a virtualization support called V-BOINC. V-BOINC distributes VirtualBox based VMs to volunteer hosts. The client has six components: *Client Interface* gives a GUI to manage membership of the client; *Communication component* handles the communications with the server; *Listener component* executes instructions from the server; *DepDisk* provides any dependency that is needed to execute a job; *VM Operations* handle operations regarding VMs; *Reliability component* implements a P2P check-pointing based reliability service. After evaluating the system for reliability and performance in comparison with Amazon EC2, the authors concluded that AdHoc Cloud is a technically feasible and can complement the current data centre based cloud computing model.

Cloud@home: is a volunteer cloud system that provides computing capabilities using aggregated capacity of computing resources from many single contributors [Cunsolo et al. 2009b]. In their proposal [Cunsolo et al. 2009a], the authors argued that Cloud@home can accommodate both the commercial and volunteer characteristics by earning a credit or selling resources and by donating the resources, respectively. A hierarchical layered architecture that has *frontend*, *virtual*, and *physical* layers is proposed. The *frontend layer* handles the management of services and resources. The *virtual layer* provides execution and storage services to the end-user. The *physical layer* provides and manages the computing resources. In order to tackle the node churn problem that occurs because of the random and unpredictable join and leave of volunteer nodes, an OpenStack⁸ based architecture is proposed [Distefano et al. 2016]. Every contributing node has a *virtualization layer* and a *Node Manager* that handle the allocation, migration, and destruction of a virtual resource. Mapping of the reference architecture of Cloud@home to Openstack and a possible technical implementation is also detailed. However, the system is not implemented and no empirical evidence is available.

Nebula: is another desktop volunteer cloud system constructed using edge resources for applications with weak performance and robustness requirements. Centralized cloud infrastructures fell short for applications with dispersed data intensive processing and do not consider locality and user contexts [Weissman et al. 2011]. Moreover, uploading large data to the centralized data centres of the current cloud infrastructures is costly. Nebula proposed to use widely distributed edge resources that are either volunteer or monetized over the Internet [Chandra and Weissman 2009]. It is a location and context-aware distributed cloud infrastructure that provides data storage and task computation. Nebula uses a combination of dedicated nodes that host global and application-specific services and volunteer nodes that donate their computational resources [Ryden et al. 2014]. Nebula has four components: *Nebula Central*, *DataStore*, *Compute Pool*, and *Nebula Monitor*. The *Nebula Central* is the first port of communication between the Nebula system and users. The *DataStore* provides per-application data storage service. The *Compute Pool* is a set of volunteer nodes

⁸<https://www.openstack.org/>

that provide computation services. The *Nebula Monitor* does a dynamically updated monitoring and characterize the volunteer nodes and the network. Computation in Nebula is performed by compute nodes within the native client sandbox provided by Chrome browser. This sandboxing provides multifaceted uses such as security and makes it easier for users to join Nebula as Chrome is a widely popular browser. Fault tolerance is achieved by redundancy in DataStore nodes and a heartbeat based mechanism is implemented to detect node failure. Load balancing and location awareness are implemented by the scheduler. The scheduler assigns tasks to a node from a set of suitable tasks that it maintains for the node. Unlike the embarrassingly parallel nature of tasks in VC, custom built subsystems help Nebula to run tasks that are tightly coupled. Nebula is evaluated together with BOINC and BOINC-MR for performance and due to the decentralized nature of the data store, Nebula outperforms the other two.

UnaCloud: is an opportunistic Infrastructure as a Service (IaaS) cloud computing model based on the preexisting computing resources within an institution/organization [E. Rosales and Vilamizar 2011; Osorio et al. 2012]. The UnaCloud follows a client/server architecture, where the server hosts a web application that works as the main interface for all UnaCloud services. The client is a light weight application, based on the design concept of SETI@Home agent, installed on each node in the system. The server is comprised of three layers: *Interface layer*, an access point to all services of UnaCloud; the *Core layer* processes user requests and responds with results; the *External layer* handles all communication tasks between the sever and clients. The Client has two layers: the *External layer* that handles communication and the *Core layer* that is responsible for task execution and related processes. In UnaCloud all the resources are homogeneous where each client can handle one VM at a time in order to avoid resource competition. Using an experimental setting with VMware based 35 clients, the authors claimed that UnaCloud is interoperable, efficient, scalable, and non-intrusive.

Similar volunteer desktop cloud systems include: *Community Cloud Computing (C3)*, a distributed community cloud created from idle resources of PCs to form a virtual data centre [Briscoe and Marinos 2009]; *Personal Cloud*, a datacenterless IaaS system that uses idle computing resources at the edge [Hari et al. 2012]; *Seattle*, an educational research platform that depends on resources denoted by users [Cappos et al. 2009]; *Clouds@home*, a BOINC based prototype of volunteer cloud system [Anjomshoa and Salleh 2014]; *DIaaS (Infrastructure as a Service with Desktops)*, an XML based system that provides fast work distribution, computing, and storage services using desktop computers [Kim et al. 2017]. A. E.S. Ahmed et al. proposed an OpenStack based hybrid IaaS private cloud that uses both dedicated and volunteer nodes for service provisioning [Ahmed et al. 2013].

3.2.2 P2P Volunteer Clouds. P2P volunteer clouds that are created using individual peers without any central authority (server) are another class of volunteer clouds. They give the advantages of scalability and flexibility of P2P architectures blended with computational conveniences of Cloud Computing. P2P volunteer clouds also provide the much needed decrease of round-trip network latency for interactive applications. P2P clouds are suitable for some applications such as embarrassingly parallel computations, multimedia streaming, online gaming that requires high level of interactivity, etc. The following are discussions on P2P volunteer cloud systems.

P2PCS: O. Babaoglu et al. proposed a fully distributed P2P cloud system, where an overlay network is maintained in order to provide fault tolerant services. Each peer in the system installs a software daemon that handles dynamic availability of peers and maintains cohesion [Babaoglu et al. 2012]. The software daemon has two interfaces: *User interface*, where users send requests to the system, and a *node-to-node interface*, where nodes use it to communicate with each other. P2PCS has different modules: *Peer Sampling Service (PSS)* that maintains an unstructured peer-to-peer network and provides membership information to each peer; *Slicing Service (SS)* provides ranking

service and returns a set of peers ranked according to a query; *Aggregation Service (AS)* replaces a global registry to provide a system-wide parameters' information to each peer. T-Man is a gossip-based protocol, where PSS uses to maintain the overlay information. The *Dispatcher* handles the users' requests. The authors claimed that a Java based prototype of P2PCS gave encouraging initial results and suggested that a P2P based decentralized cloud infrastructure is feasible.

P³R³.O.KOM: With the aim of developing a reliable cloud system, K. Graffi et al. proposed a peer-to-peer based infrastructure as a service system [Graffi et al. 2010]. The system achieved reliability by using redundancy and resource reservation based on capacity-based peer search. Two set of peers, reservation management and resource provider peer, are deployed in the system. A Weibull distribution based life time prediction of P2P resources is used for long term reservation. Simulation based evaluation showed that the prediction allowed reservation based reliable service provision in P³R³.O.KOM.

3.2.3 Social Volunteer Clouds (SVC). One of the fundamental characteristics of volunteer computing systems is the anonymity of the volunteer hosts. This anonymity results in lack of trust among the participants of VC systems, which has a direct impact on the security of the systems. Social Volunteer Cloud is a kind of community cloud that is created from resources owned, donated, and used by users of a social networking application such as Facebook or Twitter. The trust relationships that exist among users in social network applications are used in social clouds for the common purpose of sharing computing resources. The resources that can be shared in social clouds include computing capacity, software license, data, etc. The usage of social network applications provides multifold advantages such as a single user identity authentication and an inherent trust model that can be used by VC systems. Social volunteer cloud systems are discussed below.

SOCIALCLOUD: A. Mohaisen et al. proposed a social cloud computing paradigm by using social graphs to build cloud computing services with the aim of exploiting locality of data, trustworthiness, and non-reciprocation [Mohaisen et al. 2011]. The system harnesses the trust relationships that exist in social networks, social bootstrapping graph, as a source of trust for the good behaviour of other "workers in the system". By using the adjacent nodes in the social graph, the system creates a computing overlay of SOCIALCLOUD. The system uses two schedulers. The first scheduler uses local information to decide the amount of computation to outsource to each workers and the second scheduler is used to decide the order of task outsourcing and how the outsourced tasks are processed by each workers. By using real-world social graphs, the authors claimed that the majority of nodes improved their services by using the proposed services.

SoCVC (Social Cloud for Volunteer Computing): is a social cloud system that integrates volunteer computing management within a social network platform [Chard et al. 2012a]. The SoCVC uses published social network APIs, such as Facebook's OpenGraph API, to provide functionalities to Volunteer Computing systems such as a single user identity and single point of management. The authors proposed different social algorithms that encourages the participation and contribution of volunteers, such as *social score* to indicate user's social reputation. A different class of users, called *compute magnates*, is used to encourage users to contribute and draw possible resource contributors to the SoCVC. The SoCVC is implemented using Facebook and BOINC as the social app and the VC platform respectively. Simulation based experiments showed that using social algorithms has the benefit of encouraging volunteers to donate more resources.

Similar SVC systems are discussed in [Kuada and Olesen 2011] and RenderWeb 2.0 [McMahon and Milenkovic 2010]. A Social cloud system that is based on Seattle system [Cappos et al. 2009] to access users' social networks is discussed in [Caton et al. 2014].

3.2.4 Volunteer Storage Clouds. The massive increase in the amount of data collected and stored by organizations and the explosion of user generated data due to Web 2.0 applications such as social

networks and blogging sites make storage infrastructures in high demand. Some of the commercial solutions for this demand include: Amazon's Simple Storage Service (S3), which is a centralized cloud infrastructure; Wuala, a P2P cloud infrastructure; and Content Distribution Network (CDN), which offers distributed storage services using servers at the edge of the Internet. Apart from these solutions, there are also volunteer storage cloud systems that are discussed in the following paragraphs.

Storage@home: is a distributed storage infrastructure aimed at aggregating storage donations from tens of thousands of volunteer computers to provide back-up and sharing functionalities [Berg and Pande 2007]. The system has different components: the *storage clients*, *metadata server*, *identity server*, and *policy engine*. The *storage clients* are the volunteers that provide the storage resources. The *metadata server* provides centralized information storage, such as checksums and locations, for all files stored in the system as well as search and status check functionalities. The *identity server* provides the security and identity functionalities as well as location tracking of mobile and dynamic IP hosts. The *policy engine* communicates with the metadata server and identity server to provide storage, retrieval, transfer, and repair services. Polling by the identity server and heartbeat functions are used for discovering failures of nodes and redundancy and self-repair are used for failure recovery.

Fatman: is an archival storage system based on the spare resources of underutilized web servers to provide an enterprise-scale services [Qin et al. 2015]. Fatman utilizes the unused storage resources of tens of thousands of servers in large data centres. For example, the Baidu search engine provider⁹ in China has more than 240PB of storage and the utilization on both space and I/O is lower than 40% over the years [Qin et al. 2015]. Fatman has a client server architecture, where the server stores metadata and file data are stored via the datanode on volunteer clients. The data, which are 256MB-size blocks stored in distributed datanodes, are categorized as hot data (with replication) or cold data (Reed-Solomon based encoded data). The authors did an experiment on a real system and they claimed that Fatman improved data recovery, fault awareness, and reliability.

STACEE (Enhancing Storage Clouds using Edge Devices): is a storage cloud that is created using edge devices, such as PCs/media centres, routers, set-top boxes, etc., based on dynamic peer-to-peer resource provisioning [Neumann et al. 2011]. The usage of edge devices' capacity in STACEE helps to minimize energy consumption and maximize user engagement within the system. The system has a four layers architecture: the *Backend Interfaces Layer* provides access interfaces for applications to the data elements in STACEE; the *Services Layer* provides data management services such as replication, de-duplication, migration, encryption, and streaming; the *Adaptation Layer* is responsible for supporting the autonomic capability of enforcement and management of Service Level Agreements (SLAs) based on economic metrics, such as energy, that are specified in *Economic Indicators Layer*. A set of experiments are carried out to demonstrate how storage resources' availability and access latency are improved as well as energy costs are reduced in STACEE.

SASCloud: is a mobile adhoc cloud system, with a centralized control, that gives secure storage service in a network challenged area using Wi-Fi or Bluetooth [Noor et al. 2016]. SASCloud follows a hybrid architecture, with the *SASCloud Central Authority (CCA)* as a server with a functionality of registering mobile devices and *Registered Mobile Node (RM-Node)* as clients. The RM-Nodes can communicate with each other and create an adhoc cloud within a Wi-Fi or Bluetooth connection range. The reliability of a node for storage tasks is expressed in terms of *Performance Point (PP)*, which is based on node's past performance of tasks. SASCloud also has cloudlets that have sufficient memory for storage and enough CPU capability for handling any computationally intensive

⁹www.baidu.com/

operations. Encryption together with lightweight digital signature are used to ensure the security of the distributed content. A simulated experimental result showed that the SASCloud model is feasible.

3.3 Mobile Volunteer Computing

Due to the recent advances in processors with low power consumption, portable computers such as tablets and smartphones can handle computationally intensive applications and are poised to be the the computing platforms of the future. For example, Nvidia claimed that the computing power of their system on a chip series, Tegra 3, has more processing power than an Intel Core 2 Duo processor [Jones 2018]. Moreover, almost 50% of the worldwide population use smartphones and tablets, outnumbering the conventional Laptop and PC users [Partners 2017]. Furthermore, the power consumption of mobile devices is much lower than workstations. For example, M. Arslan et al. projected the cost for powering a powerful server in a data centre to be \$689/year [Arslan et al. 2012]. At the mean time the cost of powering a smartphone with computational power as powerful as Intel Core 2 Duo server is estimated to be \$1.33/year [Arslan et al. 2012]. In summary, their increasing computing power, fast growing number, and their proliferation coupled with power efficient design make mobile devices applicable for distributed computing such as Grid Computing [Chu and Humphrey 2004; Parmar et al. 2013] and Cloud Computing [Marinelli and Tan 2009]. The suitability of mobile devices for Volunteer Computing is also being investigated. However, unique challenges such as mobility and battery life coupled with relatively higher intermittent availability are some of the challenges that should be solved to fully utilize mobile devices for VC purposes. To exploit the increasing computing resources of mobile devices, many traditional VC system are extended to include such devices. For example, BOINC provides an Android based client.

Mobility, which is a constant change of the physical location of mobile devices, is a key factor that has an impact on the availability of the devices for VC tasks. As all mobile devices are battery powered, the battery life is one single major factor for both the hardware and software design of the devices. Like any VC systems, the volunteer tasks on mobile devices require computing resources. On top of that, the guest tasks also deplete the battery power of the mobile devices. This will have a negative impact on the QoE of the mobile (volunteer node) users. Internet connectivity for mobile devices is either using cellular networks or using WiFi connection. The latter has an impact on the battery of the device, while the former is most of the time expensive. Therefore, mobile devices based VC systems should take mobility, battery power and Internet connectivity into consideration during their design and implementation.

Mobile VC systems can be classified based on the entity that manages the volunteer tasks: *server driven* (client/server) or *user driven*(P2P) [Tapparello et al. 2015]. A *server driven* architecture is suitable for environments with stable network connectivity and a *user driven* is appropriate for environments where Internet connectivity is not stable. The following paragraphs discuss mobile VC systems.

CrowdLab: is a test-bed infrastructure that renders functionalities comparable to infrastructure-bound test-beds using volunteer mobile computing resources [Cuervo et al. 2011]. CrowdLab provides location anonymity by using decentralization of services. It uses hardware virtualization for isolated guest job execution to keep the confidentiality and integrity of the volunteers' data. The *Testbed Authority (TA)*, a centralized component, provides functionalities such as trust and registration of experiments and resource contributors. A co-located resource manager within a single mobile device, called *Device Authority (DA)*, creates federation with other DAs in a small geographical area called *site*. DAs are coordinated through the ad-hoc network that will be created among sites. A site directory (s-dir) is collectively maintained by site's DAs and allows a site to provide fault tolerant services by using replication. The master node will perform all the task scheduling

and role-assignment decisions periodically. From their experimentation, the authors claimed that CrowdLab provides a stable test-bed infrastructure over intermittently available resources.

GEMCloud (Green Energy Mobile Cloud): is a prototype mobile computing system that utilizes energy efficient portable devices (e.g., smartphones and tablets) connected to the Internet as computing resources [Ba et al. 2013]. GEMCloud follows a client/server architecture, where the server simply assigns the next task in a pre-determined list to the next available mobile client. The tasks are CPU-intensive computing jobs, which can be split into multiple independent tasks. Experimentation showed that even if all mobile devices are slower than the workstation competitors, they have better energy efficiency. C. Funai et al. extends GEMCloud by allowing mobile devices to connect to the system using ad-hoc networks [Funai et al. 2014]. By removing the requirement for mobile devices to have Internet connection, through decentralized job coordinators, the architecture presented in [Funai et al. 2014] allows mobile devices to join GEMCloud. Moreover, each client in the system has two functional roles: *Task Execution Points (TEPs)* that compute the assigned tasks, and *Task Distribution Points (TDPs)* that use ad-hoc connections, such as Bluetooth or Wi-Fi Direct, to receive and distribute tasks to the TEPs. The authors demonstrated the feasibility of GEMCloud with a small time and energy overhead, through experimentation.

Computing While Charging (CWC): is a framework that uses the spare computational resources of phones only while being charged, as the devices are most probably idle during the time [Arslan et al. 2012]. CWC uses a client/server architecture, where the server partitions and distributes tasks as well as aggregates results collected from smartphones as needed. CWC uses tasks' makespan prediction to distribute tasks using three different task distribution algorithms. Three profiles are defined for the phones in order to predict their availability: *plugged*, *unplugged*, and *shutdown*. These profiles are used by the central server to schedule tasks. Two kinds of failures are defined, *online* and *offline*. In case of online failure, such as a phone is disconnected from a power outlet, a task will be migrated to another available phone that can host the task. In case of offline failure, a keep-alive message will be sent to the client and if no response is received, the client will be considered as failed and the task will be migrated. A prototype based experimentation on a test-bed of 18 Android phones was used to proof the feasibility and efficiency of the system.

Serendipity: allows a mobile device to utilize the computing capacity found in other mobile devices around its surrounding to speed-up computation and save battery energy while taking the intermittent connectivity among these entities as a norm [Shi et al. 2012]. Serendipity follows a fully decentralized architecture, where mobile devices communicate with each other and resource requesters and resource providers are all mobile devices as well. Each Serendipity node has a *job engine* process, a *master* process, and several *worker* processes. The *job engine* is primarily responsible for disseminating tasks and scheduling task executions for the local master. The *master* handles the monitoring of task executions on the workers. Three task distribution techniques are proposed: *WaterFilling*, *Computing on Dissemination (COD)*, and *Unpredictable Computing on Dissemination (upCOD)*. From an Android based implementation of the system, the authors claimed Serendipity showed substantial performance gains as well as a significant improvement on the battery life of the initiator mobile devices.

FemtoCloud: is a mobile cloud system created using a collection of dynamic and self-configuring volunteer mobile devices [Habak et al. 2015]. It follows a client/server architecture with the mobile devices and a controller. The mobile device has different components: the *User Interface Module* gets user preferences with regard to resource sharing; the *Capability Estimation Module* has the task of estimating the computational capacities of a mobile device; the *User Profiling Module* builds a user profile that indicates the presence time of Femtocloud users based on gathered preferences

and behaviours in different scenarios. The controller has different components: the *Execution Prediction Module* that estimates the time of execution of a task; the *Presence Time Prediction Module* that estimates the presence time of new users and updates the estimate over time; the *Task Assignment and Scheduling Module* assigns tasks to mobile devices; *Discovery Module* discovers the available mobile devices that can join Femtocloud; the *Local Connectivity Estimation Module* estimates the available bandwidth between the computing mobile devices and the control device. After getting the computational capability estimates from the Femtocloud client service, the controller formulates a task execution schedule. Three heuristics are deployed to schedule a task on mobile devices. Android based FemtoCloud prototype performance is compared with an oracle in two different scenarios, the first managed to achieve more than 85% of what the latter managed to achieve .

Similar mobile volunteer cloud systems include: *Mobile Device Clouds*, where a set of mobile devices cooperate to offload computations [Mtibaa et al. 2013]; *CellCloud*, a mobile cloud that runs over a cellular network with a reputation-based economic incentive model [Noor et al. 2014].

AnonySense: is a privacy-aware opportunistic sensing application based on personal mobile devices to collect environmental data with in a target region [Cornelius et al. 2008]. Applications of such sensing systems range from collecting traffic or pollution reports to inferring coffee-shop space availability. The location and other context information of a volunteer node can be inferred in such applications, hence privacy concerns. AnonySense protects the privacy of volunteer nodes by providing anonymous tasking and reporting mechanism. The system has different components. The *Registration Authority (RA)* handles the registration and certification of volunteer nodes. The *Task Service (TS)* handles task related processes such as receiving task descriptions from applications, consistency checking, and task distribution to mobile nodes. The *Report Service (RS)* provides message management services such as receiving messages from mobile nodes, message aggregation, and privacy services. The communication channel is anonymized by *Mix Network (MIX)*. Applications in AnonySense use an expressive language, *AnonyTL*, to specify their tasks. After evaluating the implemented system using two application scenarios, the authors claimed that such sensing application are acceptable as they can provide privacy aware services with little overhead.

Similar opportunistic mobile sensing VC systems include PRISM [Das et al. 2010], Mobiscope [Agapie et al. 2008], LiveCompare [Deng and Cox 2009], Micro-blog [Gaonkar et al. 2008], Bubble-sensing [Lu et al. 2010], and AirShower@home [Gordienko et al. 2015].

3.4 Parallel Volunteer Computing

Nowadays, dealing with datasets in the order of terabytes or even petabytes is not uncommon. Processing such huge data needs efficient tools and algorithms. For example, Hadoop MapReduce [Dean and Ghemawat 2010] is a software framework for big data processing that has rapidly become the de facto standard. Most VC systems support embarrassingly parallel tasks that need no or little communication among the tasks. However, parallel applications, which needs heavy communication among the tasks, based on MPI, MapReduce, or other platforms and libraries are being executed on Volunteer Computing infrastructures. The following are VC systems that are specifically designed for such applications.

MPIWS: is a general framework for the execution of MPI based parallel applications using volunteer machines on the Internet [Calderón et al. 2012]. A two-level structure is proposed for MPIWS to handle the computation and communication processes. For the first level of computational part, resources are provided by volunteer computers and for the second level, the stub cluster provides services such as communication, synchronization, file access, etc. A lightweight software will be installed at the client side. An MPI installation is not required on the volunteer nodes. The

proposed model is evaluated using a prototype system with a preprocessor and a library that handles remote communication. The authors claimed that the experimental results demonstrated the feasibility and viability of MPIWS.

GiGi-MR: is a framework for the execution of MapReduce tasks using volunteer resources that are connected to the Internet [Costa et al. 2012]. GiGi-MR uses a client/server architecture coupled with a P2P communication among the volunteer hosts. It supports client-to-client transfers for scalability and replication and checkpointing are used for fault tolerance. A layer, called *MapReduce VC layer*, manages the execution of MapReduce jobs on the volunteer nodes. A group of volunteers called *mappers* first request a task from the server and download the required data from the data server. Another group of volunteers called *reducers* perform the reduce operation. The *reducers* get their inputs from the *mappers* directly, without the need to communicate with the server. The scheduler uses proximity and resource availability to select suitable mappers and reducers. Counting Attenuated Bloom Filter [Fan et al. 2000] is used to represent node's capability such as resources, applications, and services that it can offer. The turnaround time and server network traffic is measured in three MapReduce application scenarios, in order to evaluate the BOINC based prototype system [Costa et al. 2012][Costa et al. 2011]. The authors claimed that each MapReduce jobs execution time is down by half.

Adoop: is an optimized adaptation of Hadoop's MapReduce for its use in volunteer (ad hoc) cloud environments [Hamdaqa et al. 2015]. It is a history-based scheduling approach to MapReduce, where the availability history of each node affects its scheduling decisions. Adoop maintains each node's availability and current hardware utilization based score and uses this score to dynamically assign tasks to nodes. Deploying Hadoop in a highly heterogeneous, intermittently available resources like volunteer cloud is contrary to the implicit assumptions of the system. The main objective of Adoop is then to allow Hadoop to overcome the aforementioned ad hoc cloud challenges, without huge degradation in performance. The system categorizes available nodes into two groups: *Super Nodes* that are nodes with availability score higher than or equal to the configurable threshold parameter and *Normal Nodes*. A two-level scheduling approach, one based on data locality and another one based on node availability type, is used. The original Hadoop implementation is modified according to the design goals of Adoop and tested on EC2 based testbed. The authors claimed that Adoop performs better than Hadoop, in the simulated ad hoc environment, because of its self-adapting scheduler that takes into consideration the run-time utilization of the worker nodes and corresponding utilization histories.

CloudFinder: is a federation of underutilized volunteer private clouds [Rezgui et al. 2018]. It is developed with the aim of creating a computing cluster dynamically using volunteered and federated cloud resources for big data workload. CloudFinder has three major components: the command line interface based job submission system; the job distribution optimization engine that devises an optimized plan for the execution of big data workloads; the execution controller that performs the reservation of resources, distribution of data, and execution of tasks. A GENI (Global Environment for Network Innovations) based cloud federation is used to evaluate the performance of CloudFinder using a big data benchmark called HiBenchmark. The authors claimed that CloudFinder can locate optimal placements of big data workload for efficient execution.

Similar parallel VC systems include: *BOINC-MR*, a distributed VC system designed to execute embarrassingly parallel tasks, such as MapReduce, over unreliable and unsecured Internet infrastructure [Costa et al. 2011]; *MOON (MapReduce On opportunistic eNvironments)*, an extension of Hadoop in response to the bad task and data replication schema of MapReduce implementations for resources with high unavailability [Lin et al. 2010b]; *SCOLARS (Scalable COMplex LARGE Scale Volunteer Computing)*, a platform that enables the execution of MapReduce applications over the large-scale Internet [Costa et al. 2013]; *freeCycles*, a MapReduce enabled and BOINC compatible

VC system that uses Bittorrent for efficient data distribution [Bruno and Ferreira 2014]; *Hyrax*, a MapReduce implementation for Cloud Computing on Mobile Devices [Marinelli and Tan 2009]; *ADAPT* (*Availability-aware DATA PlacemenT*), which tries to improve MapReduce applications' performance in a non-dedicated environment without extra storage cost [Jin et al. 2012]; *VolpexPyMPI*, MPI library for Volunteer Computing that is implemented using Python [LeBlanc et al. 2010].

4 FUTURE OF VOLUNTEER COMPUTING

As the number of users and devices connected to the Internet increase, the need for high computing resources is on the rise as well. Moreover, Internet users are shifting from being data consumers to data producers, which results in an exponential data production increase at the edge of the Internet. This, coupled with sophisticated scientific experiments' and business intelligence applications' need for huge computation and storage resources, increases the demand for high computing resources dramatically in recent years. HPC sites and large cloud data centres have tried to cater this huge demand. However, their cost and round trip network latency due to their centralized architecture are becoming a hurdle for many users and applications. Volunteer Computing is a cheaper computing infrastructure as it is based on the unused spare computing resources of volunteer machines. Moreover, VC systems deployment proximity towards the edge of the Internet helps them to reduce the round-trip network latency significantly. Therefore, Volunteer Computing can be a viable alternative infrastructure that can complement the centralized computing infrastructures. Volunteer Computing has been studied for more than two decades. Nevertheless, its use is dominantly confined to the scientific community and the participating volunteer nodes are just extremely small proportion of the potential volunteers. The following paragraphs discuss the future direction of research and development in Volunteer Computing.

4.1 Volunteer Computing for IoT

Since its existence in the 1940's, the term "computer" has been redefined several times by researchers over the years. In 1984, we had "The network is the computer" by John Gage of Sun Microsystems; then in 2008, we had "The data centre is the computer" by David Patterson of UC Berkeley followed by "The Cloud is the computer" [Hwang et al. 2012]. Currently, we are entering the "post-cloud era" [Shi and Dustdar 2016]. In this "post-cloud-era", the major shift in the computing landscape is due to the Internet of Things (IoTs), which is the digital interconnection of everyday objects with the Internet [Conner 2010]. This paradigm shift results in the explosion of devices connected to the Internet. According to an estimate [Evans 2011], the number of IoT devices connected to the Internet will reach 50 billion by 2020. These devices will produce a huge amount of data that should be processed, stored, and transmitted and they require a computing facility that can provide fast and efficient responses. So far, Cloud Computing have been providing the necessary computing infrastructures for applications. However, the current centralized cloud infrastructures will not suffice for IoT's network intensive applications with very fast response requirements. Some of the problems of using centralized clouds to process IoT data includes network round trip latency, delay due to network congestion, and collisions of signal.

One of the solutions to this problem is moving the cloud infrastructure closer to the users/devices by creating mini data centres or using devices at the edge of the Internet. This solution is called edge/fog computing. The edge/fog proposal tries to create a middle layer cloud system so that part or whole of a computational task can be performed at the edge of the network instead of entirely in the centralized cloud data centres. Such systems that can process data produced by IoT are proposed in [Mohan and Kangasharju 2016].

Volunteer edge/fog cloud computing systems that are based on donated spare resources of devices at the edge of the Internet are also proposed. Edge-centric computing cloud proposed a consolidation of donated spare computing resources from devices such as portable devices, desktop computers, and nano data centres as a cloud [Garcia Lopez et al. 2015]. S. Alonso-Monsalve et al. studied a deployment of fog computing by using participating devices, such as PCs, smartphones or smart TVs, at the edge of the network [Alonso-Monsalve et al. 2017]. Edge devices for volunteer computing is also studied in [Alonso-Monsalve et al. 2017]. Aura, a lightweight ad hoc cloud built using IoT devices, such as smart thermostats and LED light bulbs, is investigated in [Hasan et al. 2015]. The idea of using Volunteer Computing to cater the computational needs of IoT applications is very interesting. Therefore, the use of Volunteer Computing for large IoT ecosystems is one of the research topics that will dominate the research landscape of VC in the future.

As of December 2018, more than 4.3 billion¹⁰ people are connected to the Internet. BOINC, which is a largest volunteer platform with more than 60 official and unofficial VC projects, has only 4.5 million¹¹ volunteers. This is a very insignificant proportion of the total potential volunteers on the Internet. Attracting new volunteer users and devices as well as retaining the already volunteered ones is one future direction with regard to VC research. Increasing the number of volunteers can include [Anderson 2010]: taking advantage of the trust relationships in social networks like Twitter for volunteering and bundling of VC systems with computer hardware or software (For example, Folding@home is bundled with the Sony Playstation 3); diversifying volunteer devices to be used as volunteer nodes; etc. Desktop computers and mobile devices have been the dominant devices where VC systems base their computing resource infrastructures. A new set of devices, such as our day-to-day devices like smart TVs, game consoles, refrigerators, etc., have computing resources that can be used for VC. There are some efforts to use these devices for Volunteer Computing. For example, there is a study to use smart TV sets for volunteer computing [Nakanishi 2016] or IoT devices such as LED light bulbs and smart thermostats [Hasan et al. 2015]. The full potential and challenges of using IoT devices for VC purposes and a more efficient usage of conventional devices should be studied further.

4.2 Containers Vs. Virtualization

Virtualization technologies such as VMware and Virtualbox have been used in VC systems, together with sandboxing, to ensure security and to provide isolation and portability. However, virtualization comes with performance and memory penalties, as well as I/O performance degradation usually incurred by the additional layer of indirection. Virtual Machine based applications performance penalty is much more pronounced for resource constrained systems such as VC systems that runs on spare resources of computers. Lightweight virtualization approaches are proposed to remedy the problems of virtualization technologies. Container technologies such as Docker [Docker 2018] and LXD [Canonical 2018] and Unikernel such as OSv [Kivity et al. 2014] are lightweight virtualization technologies. The efficient use of available resources and a tight integration with the host operating system make container technologies a plausible choice for VC [Herrera-Izquierdo and Grob 2017; Preeth et al. 2015]. The usage of container technologies for VC is studied in the context of edge cloud [Ismail et al. 2015]. More research is important to investigate the suitability and efficiency of container technologies for VC, specifically for volunteer cloud systems.

¹⁰<http://www.internetworldstats.com/stats.htm>

¹¹<https://boincstats.com/>

4.3 QoS in Volunteer Computing

Quality of Service (QoS) is an important issue that has been researched extensively in distributed systems. Volunteer Computing systems generally provide best effort services due to the intermittent availability and resource constraints of volunteer nodes as well as their non-intrusive characteristics. This creates a problem to provide QoS guaranteed services. Different VC systems have tried to integrate QoS from their design up-to implementation. For instance, S. Distefano et al. applied formalism in application design using non-Markovian stochastic PetriNet for QoS assessment in mobile crowdsensing applications [Distefano et al. 2015]. Oracle based task distribution and processing is also investigated for ensuring QoS of VC applications. For example, Reservation in Advance (RA) and Meta-Scheduling in Advance (MSA) algorithms are proposed in [Conejero et al. 2014]. Some VC systems tried to enhance Quality of Experience (QoE) of users by using a hybrid system of dedicated infrastructure with volunteer infrastructure, where the user request will be directed to the dedicated infrastructure when it cannot be met by the volunteer infrastructure [Delamare et al. 2014; Urbah et al. 2009]. FriendBox [Moreno-Martínez et al. 2012] uses the storage resources of friends in a social network setting together with public storage cloud for rendering storage services. E. Miluzzo et al. discussed a volunteer system that uses a cloud infrastructure in the case of shortage of resources in volunteer infrastructure [Miluzzo et al. 2012].

More research is needed towards providing quality guaranteed services in VC systems in order to increase their adoption for general applications. Prediction based techniques that predict system behaviour under a variety of conditions can be applied for ensuring QoS in VC systems [Mengistu et al. 2018b]. However, different prediction techniques should be investigated and pros and cons of the techniques with regard to Volunteer Computing should be known. Different machine learning techniques have been deployed to ensure QoS in distributed systems, such as reinforcement learning, regression analysis, Artificial Neural Networks, etc [Karniavoura and Magoutis 2019]. The suitability and application of machine learning and deep learning techniques for VC systems is another research direction that needs further investigation.

4.4 Blockchain for Volunteer Computing

In recent years, digital currencies such as Bitcoin are becoming very popular with Bitcoin capital market surpassing \$4 billion in 2017 [blockchain.info 2017]. These digital currencies are based on blockchain technology. Blockchain, is a distributed electronic ledger system that holds a complete list of transaction records, as sequence of blocks, like conventional public ledger [Chuen 2015]. It is a combination of multiple technologies such as asymmetric cryptography, P2P networking, distributed consensus, and an economic model. Blockchain can be public, private, or consortium where a node with higher authority can be preselected [Lin and Liao 2017]. Some of the characteristics of blockchain includes decentralization, transparency, autonomy, anonymity, auditability, and immutability. These characteristics make blockchain technology attractive in areas such as financial services, healthcare, smart contracts, IoTs, etc. [Zheng et al. 2017]. Despite its popularity and success, the blockchain technology has some technical challenges such as scalability and security problems as well as huge electric energy consumption for the number crunching.

Blockchain, can be applied in volunteer computing as a platform that allows participants to donate and use computing resources in a distributed manner and make money in the process. The distributed ledger system can be used as a secure way of keeping track of what is donated and what is consumed. The decentralized nature, persistence, together with immutability characteristics make blockchain attractive for business modelling in Volunteer Computing systems. The Proof of Work (PoW) and Proof of Stake (PoS) can be translated to reputations that will be stored

in blockchain of VC systems. The applicability of blockchain technology for VC should be investigated further.

5 CONCLUSION

The rapid increase of the number of devices connected to the Internet due to the Internet of Things and the massive increase in the amount of data collected and stored by organizations coupled with the explosion of user generated data due to Web 2.0 applications such as social networks and blogs make the need for high computing infrastructures increase. Supercomputing sites, HPC infrastructures, and large Cloud Computing data centres have been used to cater the much needed computing infrastructures. In addition, Volunteer Computing is becoming a cheaper and greener alternative computing infrastructure that can complement these dedicated, centralized, and expensive data infrastructures. The aggregated idle computing resources of devices ranging from desktop computers to routers and smart TVs are being used for providing computing infrastructures for resource intensive tasks such as scientific simulations and big data analysis. In order to harness the full potentials of Volunteer Computing, we need to devise new techniques and algorithms as well as improve the already existing ones. This paper discussed existing Volunteer Computing systems and their technical details together with their taxonomical classification. Moreover, the paper sheds some light on the future research directions of Volunteer Computing systems.

REFERENCES

- C Adam-Bourdarios, D Cameron, A Filipi, E Lancon, W Wu, and ATLAS Collaboration. 2015. ATLAS@Home: Harnessing Volunteer Computing for HEP. *Journal of Physics: Conference Series* 664, 2 (2015), 022009.
- Atul Adya, William J. Bolosky, Miguel Castro, Gerald Cermak, Ronnie Chaiken, John R. Douceur, Jon Howell, Jacob R. Lorch, Marvin Theimer, and Roger P. Wattenhofer. 2002. Farsite: Federated, Available, and Reliable Storage for an Incompletely Trusted Environment. *SIGOPS* 36, SI (Dec. 2002), 1–14.
- E. Agapie, G. Chen, D. Houston, E. Howard, J. Kim, M. Y. Mun, A. Mondschein, S. Reddy, R. Rosario, J. Ryder, A. Steiner, J. Burke, E. Estrin, M. Hansen, and M. Rahimi. 2008. Seeing Our Signals: Combining Location Traces and Web-based Models for Personal Discovery. In *Proceedings of the 9th Workshop on Mobile Computing Systems and Applications (HotMobile '08)*, 6–10.
- A. E. S. Ahmed, A. K. Alsammak, and E. Algizawy. 2013. A New Approach to Manage and Utilize Cloud Computing Underused Resources. *International Journal of Computer Applications* 76, 11 (2013), 29–36.
- Myles Allen. 1999. Do-it-yourself climate prediction. *Nature* 401, 642 (Oct. 1999).
- S. Alonso-Monsalve, F. Garcia-Carballeira, and A. Calderón. 2017. Fog computing through public-resource computing and storage. In *2017 Second International Conference on Fog and Mobile Edge Computing (FMEC)*, 81–87.
- D. P. Anderson. 2004. BOINC: a system for public-resource computing and storage. In *Fifth IEEE/ACM International Workshop on Grid Computing*, 4–10.
- David P. Anderson. 2010. Volunteer Computing: The Ultimate Cloud. *Crossroads* 16, 3 (March 2010), 7–10.
- D. P. Anderson. 2011. Emulating Volunteer Computing Scheduling Policies. In *2011 IEEE International Symposium on Parallel and Distributed Processing Workshops and Phd Forum*, 1839–1846.
- David P. Anderson, Jeff Cobb, Eric Korpela, Matt Lebofsky, and Dan Werthimer. 2002. SETI@Home: An Experiment in Public-resource Computing. *Commun. ACM* 45, 11 (Nov. 2002), 56–61.
- David P. Anderson, Eric Korpela, and Rom Walton. 2005. High-Performance Task Distribution for Volunteer Computing. In *Proceedings of the First International Conference on e-Science and Grid Computing (E-SCIENCE '05)*. IEEE Computer Society, Washington, DC, USA, 196–203.
- D. P. Anderson and J. McLeod. 2007. Local Scheduling for Volunteer Computing. In *2007 IEEE International Parallel and Distributed Processing Symposium*, 1–8.
- N. Andrade, Brasileiro, F., Mowbray, M., and W Cirne. 2009. *A Reciprocation-Based Economy for Multiple Services in a Computational Grid, in Market-Oriented Grid and Utility Computing*. John Wiley and Sons Inc., NJ, USA. 357–370 pages.
- Nazareno Andrade and Walfredo Cirne. 2005. Peer-to-peer grid computing with the OurGrid Community.
- Nazareno Andrade, Walfredo Cirne, Francisco Brasileiro, and Paulo Roisenberg. 2003. OurGrid: An Approach to Easily Assemble Grids with Equitable Resource Sharing. In *Job Scheduling Strategies for Parallel Processing*, Dror Feitelson, Larry Rudolph, and Uwe Schwiegelshohn (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 61–86.

- M. Anjomshoa and M. Salleh. 2014. Overview on Clouds@home: Virtualization Mechanism for Volunteer Computing. In *PDOS 2014 International Conference Parallel and Distributed Computing Systems*. 11–19.
- Mustafa Y. Arslan, Indrajeet Singh, Shailendra Singh, Harsha V. Madhyastha, Karthikeyan Sundaresan, and Srikanth V. Krishnamurthy. 2012. Computing While Charging: Building a Distributed Computing Infrastructure Using Smartphones. In *Proceedings of the 8th International Conference on Emerging Networking Experiments and Technologies (CoNEXT '12)*. 193–204.
- H. Ba, W. Heinzelman, C. A. Janssen, and J. Shi. 2013. Mobile computing - A green computing resource. In *2013 IEEE Wireless Communications and Networking Conference (WCNC)*. 4451–4456.
- Ozalp Babaoglu, Moreno Marzolla, and Michele Tamburini. 2012. Design and Implementation of a P2P Cloud System. In *Proceedings of the 27th Annual ACM Symposium on Applied Computing*. 412–417.
- A. L. Beberg and V. S. Pande. 2007. Storage@home: Petascale Distributed Storage. In *2007 IEEE International Parallel and Distributed Processing Symposium*. 1–6.
- blockchain.info. 2017. The Estimated Transaction Value in USD value. (2017). <https://blockchain.info/charts/estimated-transaction-volume-usd?daysAverageString=7×pan=all>
- F. Brasileiro, E. Araujo, W. Voorsluys, M. Oliveira, and F. Figueiredo. 2007. Bridging the High Performance Computing Gap: the OurGrid Experience. In *Seventh IEEE International Symposium on Cluster Computing and the Grid (CCGrid '07)*. 817–822.
- G. Briscoe and A. Marinos. 2009. Digital ecosystems in the clouds: Towards community cloud computing. In *2009 3rd IEEE International Conference on Digital Ecosystems and Technologies*. 103–108.
- David C. Brock. 2006. *Understanding Moore's Law: Four Decades of Innovation*. Philadelphia: Chemical Heritage Press.
- RE Brown, ER Masanet, B Nordman, WF Tschudi, A Shehabi, J Stanley, JG Koomey, DA Sartor, and PT Chan. 2008. Report to congress on server and data center energy efficiency: Public law 109-431. (2008).
- Rodrigo Bruno and Paulo Ferreira. 2014. freeCycles: Efficient Data Distribution for Volunteer Computing. In *Proceedings of the Fourth International Workshop on Cloud Data and Platforms (CloudDP '14)*. Article 4, 4:1–4:6 pages.
- A. R. Butt, T. A. Johnson, Yili Zheng, and Y. C. Hu. 2004. Kosha: A Peer-to-Peer Enhancement for the Network File System. In *Supercomputing, Proceedings of the ACM/IEEE SC2004 Conference*. 51–51.
- R. Buyya and S. Venugopal. 2004. The Gridbus toolkit for service oriented grid and utility computing: an overview and status report. In *1st IEEE International Workshop on Grid Economics and Business Models, 2004. GECON 2004*. 19–66.
- Alejandro Calderón, Felix García-Carballeira, Borja Bergua, Luis Miguel Sánchez, and Jesús Carretero. 2012. Expanding the Volunteer Computing Scenario: A Novel Approach to Use Parallel Applications on Volunteer Computing. *Future Gener. Comput. Syst.* 28, 6 (June 2012), 881–889.
- Paulo Picota Cano and Miguel Vargas-Lombardo. 2012. Security Threats. Volunteer computing environments using the Berkeley Open Infrastructure for Network Computing (BOINC). *Int.J.Computer Technology and Applications* 3 (2012), 944–948.
- Ltd Canonical. 2018. LXD. (2018). <https://linuxcontainers.org/lxd/introduction/>
- Jiannong Cao, Oscar M. K. Kwong, Xianbing Wang, and Wentong Cai. 2004. *A Peer-to-Peer Approach to Task Scheduling in Computation Grid*. Springer Berlin Heidelberg, Berlin, Heidelberg, 316–323.
- Franck Cappello, Samir Djilali and Gilles Fedak, Thomas Herault, Frédéric Magniette, Vincent, and Néril Oleg Lodygensky. 2005. Computing on large-scale distributed systems: XtremWeb architecture, programming models, security, tests and convergence with grid. *Future Generation Computer Systems* 21 (March 2005), 417–437.
- Justin Cappos, Ivan Beschastnikh, Arvind Krishnamurthy, and Tom Anderson. 2009. *Seattle: A platform for educational cloud computing*. 111–115.
- H. Castro, E. Rosales, M. Villamizar, and A. Jiménez. 2010. UnaGrid: On Demand Opportunistic Desktop Grid. In *2010 10th IEEE/ACM International Conference on Cluster, Cloud and Grid Computing*. 661–666.
- S. Caton, C. Haas, K. Chard, K. Bubendorfer, and O. F. Rana. 2014. A Social Compute Cloud: Allocating and Sharing Infrastructure Resources via Social Networks. *IEEE Transactions on Services Computing* 7, 3 (July 2014), 359–372.
- Abhishek Chandra and Jon Weissman. 2009. Nebulas: Using Distributed Voluntary Resources to Build Clouds. In *Proceedings of the 2009 Conference on Hot Topics in Cloud Computing (HotCloud'09)*. USENIX Association, Article 2.
- K. Chard, K. Bubendorfer, S. Caton, and O. F. Rana. 2012b. Social Cloud Computing: A Vision for Socially Motivated Resource Sharing. *IEEE Transactions on Services Computing* 5, 4 (Fourth 2012), 551–563.
- R. Chard, K. Bubendorfer, and K. Chard. 2012a. Experiences in the design and implementation of a Social Cloud for Volunteer Computing. In *2012 IEEE 8th International Conference on E-Science*. 1–8.
- Dunren Che and Wen-Chi Hou. 2011. A Novel “Credit Union” Model of Cloud Computing. In *Digital Information and Communication Technology and Its Applications*, Hocine Cherifi, Jasni Mohamad Zain, and Eyas El-Qawasmeh (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 714–727.
- Andrew Chien, Brad Calder, Stephen Elbert, and Karan Bhatia. 2003. Entropia: architecture and performance of an enterprise desktop grid system. *J. Parallel and Distrib. Comput.* 63, 5 (2003), 597 – 610.

- Sungjin Choi, Hongsoo Kim, Eunjung Byun, and Chongsun Hwang. 2006b. *A Taxonomy of Desktop Grid Systems Focusing on Scheduling*. Technical Report.
- Sung Jin Choi, MaengSoon Baik, JoonMin Gil, ChanYeol Park, SoonYoung Jung, and ChongSun Hwang. 2006a. Group-based dynamic computational replication mechanism in peer-to-peer grid computing. In *Sixth IEEE International Symposium on Cluster Computing and the Grid. CCGRID 06.*, Vol. 2.
- D. C. Chu and M. Humphrey. 2004. Mobile OGSINET: grid computing on mobile devices. In *Fifth IEEE/ACM International Workshop on Grid Computing*. 182–191.
- X. Chu, K. Nadiminti, C. Jin, S. Venugopal, and R. Buyya. 2007. Aneka: Next-Generation Enterprise Grid Platform for e-Science and e-Business Applications. In *Third IEEE International Conference on e-Science and Grid Computing (e-Science 2007)*. 151–159.
- D. Lee Kuo Chuen. 2015. *Handbook of Digital Currency* (1st ed.). Elsevier, New York, NY.
- Javier Conejero, Blanca Caminero, Carmen Carrión, and Luis Tomás. 2014. From volunteer to trustable computing: Providing QoS-aware scheduling mechanisms for multi-grid computing environments. *Future Generation Computer Systems* 34 (2014), 76 – 93.
- Margery Conner. 2010. Sensors empower the Internet of Things. (May 2010). <https://www.gartner.com/newsroom/id/703807>
- Seth Cooper, Firas Khatib, Adrien Treuille, Janos Barbero, Jeehyung Lee, Michael Beenen, Andrew Leaver-Fay, David Baker, Zoran Popovi, and Foldit players. 2007. Predicting protein structures with a multiplayer online game. *Nature* 466, 756 (2007).
- Cory Cornelius, Apu Kapadia, David Kotz, Dan Peebles, Minho Shin, and Nikos Triandopoulos. 2008. AnonymSense: Privacy-aware People-centric Sensing. In *Proceedings of the 6th International Conference on Mobile Systems, Applications, and Services (MobiSys '08)*. 211–224.
- Fernando Costa, João Nuno Silva, Luís Veiga, and Paulo Ferreira. 2012. Large-scale volunteer computing over the Internet. *Journal of Internet Services and Applications* 3, 3 (01 Dec 2012), 329–346.
- F. Costa, L. Silva, and M. Dahlin. 2011. Volunteer Cloud Computing: MapReduce over the Internet. In *IEEE International Symposium on Parallel and Distributed Processing Workshops and Phd Forum*. 1855–1862.
- Fernando Costa, Luis Veiga, and Paulo Ferreira. 2013. Internet-scale support for map-reduce processing. *Journal of Internet Services and Applications* 4, 1 (12 Nov 2013), 18.
- E. Cuervo, P. Gilbert, B. Wu, and L. P. Cox. 2011. CrowdLab: An architecture for volunteer mobile testbeds. In *Third International Conference on Communication Systems and Networks (COMSNETS 2011)*. 1–10.
- V. D. Cunsolo, S. Distefano, A. Puliafito, and Marco Scarpa. 2009a. Cloud@Home: Bridging the Gap between Volunteer and Cloud Computing. Heidelberg.
- V. D. Cunsolo, S. Distefano, A. Puliafito, and M. Scarpa. 2009b. Volunteer Computing and Desktop Cloud: The Cloud@Home Paradigm. In *2009 Eighth IEEE International Symposium on Network Computing and Applications*. 134–139.
- Radoslav Danilak. 2017. Why Energy Is A Big And Rapidly Growing Problem For Data Centers. (2017).
- Peter Darch and Annamaria Carusi. 2010. Retaining volunteers in volunteer computing projects. *Philosophical Transactions of the Royal Society of London A: Mathematical, Physical and Engineering Sciences* 368, 1926 (2010), 4177–4192.
- Tathagata Das, Prashanth Mohan, Venkata N. Padmanabhan, Ramachandran Ramjee, and Asankhaya Sharma. 2010. PRISM: Platform for Remote Sensing Using Smartphones. In *Proceedings of the 8th International Conference on Mobile Systems, Applications, and Services (MobiSys '10)*. ACM, New York, NY, USA, 63–76.
- Jeffrey Dean and Sanjay Ghemawat. 2010. Communications of the ACM. *International Journal of Network Security* 539, 1 (t 2010), 72–77.
- Simon Delamare, Gilles Fedak, Derrick Kondo, and Oleg Lodygensky. 2014. SpeQuloS: a QoS service for hybrid and elastic computing infrastructures. *Cluster Computing* 17, 1 (01 Mar 2014), 79–100.
- Linda Deng and Landon P. Cox. 2009. LiveCompare: Grocery Bargain Hunting Through Participatory Sensing. In *Proceedings of the 10th Workshop on Mobile Computing Systems and Applications (HotMobile '09)*. Article 4, 4:1–4:6 pages.
- Travis Desell, Malik Magdon-Ismail, Boleslaw Szymanski, Carlos A. Varela, Heidi Newberg, and David P. Anderson. 2010. *Validating Evolutionary Algorithms on Volunteer Computing Grids*. Springer Berlin Heidelberg, Berlin, Heidelberg, 29–41.
- Sebastian Deterding, Dan Dixon, Rilla Khaled, and Lennart Nacke. 2011. From Game Design Elements to Gamefulness: Defining "Gamification". In *Proceedings of the 15th International Academic MindTrek Conference: Envisioning Future Media Environments (MindTrek '11)*. 9–15.
- Salvatore Distefano, Francesco Longo, and Marco Scarpa. 2015. QoS Assessment of Mobile Crowdsensing Services. *Journal of Grid Computing* 13, 4 (Dec. 2015), 629–650.
- Salvatore Distefano, Giovanni Merlino, and Antonio Puliafito. 2016. An OpenStack-based Implementation of a Volunteer Cloud. *Communications in Computer and Information Science* 256 (2016), 389–403.
- Inc. Docker. 2018. Docker. (2018). <https://www.docker.com/>

- P. Domingues, P. Marques, and L. Silva. 2005. Resource usage of Windows computer laboratories. In *2005 International Conference on Parallel Processing Workshops*. 469–476.
- Daniela Dorneanu. 2011. Volunteer Computing at CERN - sustainability model proposal. (2011).
- H. Castro E. Rosales and M. Villamizar. 2011. Unacloud: Opportunistic Cloud Computing Infrastructure as a Service,. In *2011 the Second International Conference on Cloud Computing, GRIDs, and Virtualization*. 187–194.
- T. Estrada, D. A. Flores, M. Taufer, P. J. Teller, A. Kerstens, and D. P. Anderson. 2006. The Effectiveness of Threshold-Based Scheduling Policies in BOINC Projects. In *2006 Second IEEE International Conference on e-Science and Grid Computing (e-Science'06)*. 88–88.
- Trilce Estrada, Olac Fuentes, and Michela Taufer. 2008. A Distributed Evolutionary Method to Design Scheduling Policies for Volunteer Computing. *SIGMETRICS Perform. Eval. Rev.* 36, 3 (Nov. 2008), 40–49.
- Trilce Estrada, Michela Taufer, and David P. Anderson. 2009. Performance Prediction and Analysis of BOINC Projects: An Empirical Study with EmBOINC. *Journal of Grid Computing* 7, 4 (19 Aug 2009), 537.
- Dave Evans. 2011. The Internet of Things: How the Next Evolution of the Internet is Changing Everything. *CISCO Internet Business Solutions Group (IBSG)* 1 (April 2011), 1–11.
- Li Fan, Pei Cao, Jussara Almeida, and Andrei Z. Broder. 2000. Summary Cache: A Scalable Wide-area Web Cache Sharing Protocol. *IEEE/ACM Trans. Netw.* 8, 3 (June 2000), 281–293.
- Gilles Fedak. 2010. *Recent Advances and Research Challenges in Desktop Grid and Volunteer Computing*. Springer. 171–185 pages.
- G. Fedak, C. Germain, V. Neri, and F. Cappello. 2001. XtremWeb: a generic global computing system. In *Proceedings First IEEE/ACM International Symposium on Cluster Computing and the Grid*. 582–587.
- Gilles Fedak, Haiwu He, and Franck Cappello. 2009. BitDew: A Data Management and Distribution Service with Multi-protocol File Transfer and Metadata Abstraction. *J. Netw. Comput. Appl.* 32, 5 (Sept. 2009), 961–975.
- C. Funai, C. Tapparelo, H. Ba, B. Karaoglu, and W. Heinzelman. 2014. Extending volunteer computing through mobile ad hoc networking. In *2014 IEEE Global Communications Conference*. 32–38.
- Shravan Gaonkar, Jack Li, Romit Roy Choudhury, Landon Cox, and Al Schmidt. 2008. Micro-Blog: Sharing and Querying Content Through Mobile Phones and Social Participation. In *Proceedings of the 6th International Conference on Mobile Systems, Applications, and Services (MobiSys '08)*. ACM, New York, NY, USA, 174–186.
- Pedro Garcia Lopez, Alberto Montesor, Dick Epema, Anwitaman Datta, Teruo Higashino, Adriana Iamnitchi, Marinho Barcellos, Pascal Felber, and Etienne Riviere. 2015. Edge-centric Computing: Vision and Challenges. *SIGCOMM Comput. Commun. Rev.* 45, 5 (Sept. 2015), 37–42.
- Gartner. 2008. Gartner more than 1 billion pcs in use worldwide and headed to 2 billion units by 2014. (June 2008). <https://www.edn.com/design/sensors/4363366/Sensors-empower-the-quot-Internet-of-Things-quot->
- Toktam Ghafarian, Hossein Deldari, Bahman Javadi, Mohammad H. Yaghmaee, and Rajkumar Buyya. 2013. CycloidGrid: A Proximity-aware P2P-based Resource Discovery Architecture in Volunteer Computing Systems. *Future Gener. Comput. Syst.* 29, 6 (Aug. 2013), 1583–1595.
- Toktam Ghafarian and Bahman Javadi. 2015. Cloud-aware Data Intensive Workflow Scheduling on Volunteer Computing Systems. *Future Gener. Comput. Syst.* 51, C (Oct. 2015), 87–97.
- N. Gordienko, O. Lodygensky, G. Fedak, and Y. Gordienko. 2015. Synergy of volunteer measurements and volunteer computing for effective data collecting, processing, simulating and analyzing on a worldwide scale. In *2015 38th International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO)*. 193–198.
- K. Graffi, D. Stingl, C. Gross, H. Nguyen, A. Kovacevic, and R. Steinmetz. 2010. Towards a P2P Cloud: Reliable Resource Reservations in Unreliable P2P Systems. In *2010 IEEE 16th International Conference on Parallel and Distributed Systems*. 27–34.
- The Climate Group. 2008. Smart 2020: Enabling the low carbon economy in the information age, Tech. Rep. (2008).
- Huseyin Guler, B. Barla Cambazoglu, and Oznur Ozkasap. 2015. Task allocation in volunteer computing networks under monetary budget constraints. *Peer-to-Peer Networking and Applications* 8, 6 (01 Nov 2015), 938–951.
- K. Habak, M. Ammar, K. A. Harras, and E. Zegura. 2015. Femto Clouds: Leveraging Mobile Devices to Provide Cloud Service at the Edge. In *8th IEEE International Conference on Cloud Computing*. 9–16.
- Mohammad Hamdaqa, Mohamed M. Sabri, Akshay Singh, and Ladan Tahvildari. 2015. Adoop: MapReduce for Ad-hoc Cloud Computing. In *Proceedings of the 25th Annual International Conference on Computer Science and Software Engineering (CASCON '15)*. IBM Corp., Riverton, NJ, USA, 26–34.
- Adishesu Hari, Ramesh Viswanathan, T. V. Lakshman, and Y. J. Chang. 2012. The Personal Cloud: Design, Architecture and Matchmaking Algorithms for Resource Management. In *Proceedings of the 2Nd USENIX Conference on Hot Topics in Management of Internet, Cloud, and Enterprise Networks and Services*. USENIX Association, Berkeley, CA, USA, 3–3.
- M. Harvey, G. Giupponi, J. Villa'-Freixa, and G. De Fabritiis. 2007. PS3GRID.NET: Building a Distributed Supercomputer Using the PlayStation 3. *Distributed and Grid Computing - Science Made Transparent for Everyone. Principles, Applications and Supporting Communities* (Fourth 2007), 1–10.

- R. Hasan, M. M. Hossain, and R. Khan. 2015. Aura: An IoT Based Cloud Infrastructure for Localized Mobile Computation Outsourcing. In *2015 3rd IEEE International Conference on Mobile Cloud Computing, Services, and Engineering*. 183–188.
- Ferry Hendrixx, Kris Bubendorfer, and Ryan Chard. 2015. Reputation systems: A survey and taxonomy. *J. Parallel and Distrib. Comput.* 75 (2015), 184 – 197.
- Werner Herr, D I Kaltchev, E McIntosh, and F Schmidt. 2006. Large Scale Beam-beam Simulations for the CERN LHC using Distributed Computing. In *Proceedings of the 10th European Particle Accelerator Conference*,. 526–528.
- Luis Herrera-Izquierdo and Marc Grob. 2017. A performance evaluation between Docker container and Virtual Machines in cloud computing architectures. In *MASKANA, CEDIA*. 127–133.
- K. Hwang, G. Fox, , and J. Dongarra. 2012. *Distributed and Cloud Computing: From Parallel Processing to the Internet of Things*. Elsevier Inc.
- B. I. Ismail, E. Mostajeran Goortani, M. B. Ab Karim, W. Ming Tat, S. Setapa, J. Y. Luke, and O. Hong Hoe. 2015. Evaluation of Docker as Edge computing platform. In *2015 IEEE Conference on Open Systems (ICOS)*. 130–135.
- B. Javadi, D. Kondo, J. M. Vincent, and D. P. Anderson. 2011. Discovering Statistical Models of Availability in Large Distributed Systems: An Empirical Study of SETI@home. *IEEE Transactions on Parallel and Distributed Systems* 22, 11 (Nov 2011), 1896–1903.
- Hui Jin, Xi Yang, Xian-He Sun, and Ioan Raicu. 2012. ADAPT: Availability-Aware MapReduce Data Placement for Non-dedicated Distributed Computing.. In *ICDCS*. IEEE Computer Society, 516–525.
- Nicola Jones. 2018. How to stop data centres from gobbling up the worlds electricity. *Nature International Journal of Science* 561 (Sept 2018), 163–166.
- Audun Jøsang, Roslan Ismail, and Colin Boyd. 2007. A Survey of Trust and Reputation Systems for Online Service Provision. *Decis. Support Syst.* 43, 2 (March 2007), 618–644.
- F. Karniavoura and K. Magoutis. 2019. Decision-Making Approaches for Performance QoS in Distributed Storage Systems: A Survey. *IEEE Transactions on Parallel and Distributed Systems* (2019), 1–1.
- Hyun-Woo Kim, Jaekyung Han, Jong Hyuk Park, and Young-Sik Jeong. 2017. DaaS: Resource Management System for the Intra-Cloud with On-Premise Desktops. *Symmetry* 9, 1 (2017), 1–17.
- Graham N. C. Kirby, Alan Dearle, Angus Macdonald, and Alvaro A. A. Fernandes. 2010. An Approach to Ad hoc Cloud Computing. *CoRR* abs/1002.4738 (2010).
- Avi Kivity, Dor Laor, Glauber Costa, Pekka Enberg, Nadav Har’El, Don Marti, and Vlad Zolotarov. 2014. OSv—Optimizing the Operating System for Virtual Machines. In *2014 USENIX Annual Technical Conference (USENIX ATC 14)*. USENIX Association, Philadelphia, PA, 61–72.
- D. Kondo, D. P. Anderson, and J. M. Vii. 2007. Performance Evaluation of Scheduling Policies for Volunteer Computing. In *Third IEEE International Conference on e-Science and Grid Computing (e-Science 2007)*. 415–422.
- D. Kondo, A. Andrzejak, and D. P. Anderson. 2008. On correlated availability in Internet-distributed systems. In *2008 9th IEEE/ACM International Conference on Grid Computing*. 276–283.
- Klaus Krauter, Rajkumar Buyya, and Muthucumaru Maheswaran. 2002. A Taxonomy and Survey of Grid Resource Management Systems for Distributed Computing. *Software Practice and Experience*. 32, 2 (Feb. 2002), 135–164.
- Eric Kuada and Henning Olesen. 2011. Social Network Approach to Provisioning and Management of Cloud Computing Services for Enterprises. In *Proceedings The Second International Conference on Cloud Computing, GRIDS, and Virtualization*. 98–104.
- Stefan M. Larson, Christopher D. Snow, Michael Shirts, and Vijay S. Pande. 2002. *Folding@Home and Genome@Home: Using distributed computing to tackle previously intractable problems in computational biology*. Horizon Press.
- T. P. LeBlanc, J. Subhlok, and E. Gabriel. 2010. A High-Level Interpreted MPI Library for Parallel Computing in Volunteer Environments. In *2010 10th IEEE/ACM International Conference on Cluster, Cloud and Grid Computing*. 673–678.
- Josep L. Lerida, Francesc Solsona, Porfidio Hernandez, Francesc Gine, Mauricio Hanzich, and Josep Conde. 2013. State-based Predictions with Self-correction on Enterprise Desktop Grid Environments. *J. Parallel Distrib. Comput.* 73, 6 (June 2013), 777–789.
- Sarmenta LFG. 2001. Volunteer computing. Department of Electrical Engineering and Computer Science. Massachusetts Institute of Technology. (2001).
- M. Li, J. Li, and S. Xie. 2007. Posted Price Model Based on GRS and Its Optimization Using in Grid Resource Allocation. In *2007 International Conference on Wireless Communications, Networking and Mobile Computing*. 3172–3175.
- S. Lili, Y. Shoubao, G. Liangmin, and W. Bin. 2009. A Markov Chain Based Resource Prediction in Computational Grid. In *2009 Fourth International Conference on Frontier of Computer Science and Technology*. 119–124.
- Heshan Lin, Xiaosong Ma, Jeremy Archuleta, Wu-chun Feng, Mark Gardner, and Zhe Zhang. 2010b. MOON: MapReduce On Opportunistic eNvironments. In *Proceedings of the 19th ACM International Symposium on High Performance Distributed Computing (HPDC '10)*. 95–106.
- Iuon-Chang Lin and Tzu-Chun Liao. 2017. A Survey of Blockchain Security Issues and Challenges. *International Journal of Network Security* 19, 5 (Sept 2017), 653–6592.

- W. Y. Lin, G. Y. Lin, and H. Y. Wei. 2010a. Dynamic Auction Mechanism for Cloud Resource Allocation. In *2010 10th IEEE/ACM International Conference on Cluster, Cloud and Grid Computing*. 591–592.
- M. J. Litzkow, M. Livny, and M. W. Mutka. 1988. Condor-a hunter of idle workstations. In *[1988] Proceedings. The 8th International Conference on Distributed*. 104–111.
- Hong Lu, Nicholas D. Lane, Shane B. Eisenman, and Andrew T. Campbell. 2010. Fast Track Article: Bubble-sensing: Binding Sensing Tasks to the Physical World. *Pervasive Mob. Comput.* 6, 1 (Feb. 2010), 58–71.
- Akshay Luther, Rajkumar Buyya, Rajiv Ranjan, and Srikumar Venugopal. 2005. Alchemi: A .NET-based Enterprise Grid Computing System. In *6th International Conference on Internet Computing (ICOMP'05), Las Vegas*. 269–278.
- Xiaosong Ma, Sudharshan S. Vazhkudai, and Zhe Zhang. 2009. Improving Data Availability for Better Access Performance: A Study on Caching Scientific Data on Distributed Desktop Workstations. *Journal of Grid Computing* 7, 4 (16 Jul 2009), 419.
- Eugene Marinelli and Jiaqi Tan. 2009. Hyrax: Cloud Computing on Mobile Devices using MapReduce.
- G. A. McGilvary, A. Barker, and M. Atkinson. 2015. Ad Hoc Cloud Computing. In *2015 IEEE 8th International Conference on Cloud Computing*. 1063–1068.
- Adam McMahon and Victor Milenkovic. 2010. *Social volunteer computing*. Vol. 3. 268–273.
- Peter M. Mell and Timothy Grance. 2011. *The NIST Definition of Cloud Computing*. Technical Report. Gaithersburg, MD, United States.
- T. Mengistu, A. Alahmadi, A. Albuali, Y. Alsenani, and D. Che. 2017. A “No Data Center” Solution to Cloud Computing. In *2017 IEEE 10th International Conference on Cloud Computing*. 714–717.
- Tessema Mindaye Mengistu. 2018. *RESOURCE MANAGEMENT FRAMEWORK FOR VOLUNTEER CLOUD COMPUTING*. PhD dissertation. 1613. Southern Illinois University Carbondale.
- Tessema M. Mengistu, Abdulrahman M. Alahmadi, Yousef Alsenani, Abdullah Albuali, and Dunren Che. 2018a. cuCloud: Volunteer Computing as a Service (VCaaS) System. In *Cloud Computing – CLOUD 2018*, Min Luo and Liang-Jie Zhang (Eds.). Springer International Publishing, 251–264.
- T. M. Mengistu, D. Che, A. Alahmadi, and S. Lu. 2018b. Semi-Markov Process Based Reliability and Availability Prediction for Volunteer Cloud Systems. In *2018 IEEE 11th International Conference on Cloud Computing (CLOUD)*, Vol. 00. 359–366.
- Tiziano Mengotti. 2004. GPU, a framework for distributed computing over gnutella. (March 2004). <https://pdfs.semanticscholar.org/fb5b/fb4a1bb4fbc0aaec29dd05d9512670a4134f.pdf>
- Emiliano Miluzzo, Ramón Cáceres, and Yih-Farn Chen. 2012. Vision: MClouds - Computing on Clouds of Mobile Devices. In *Proceedings of the Third ACM Workshop on Mobile Cloud Computing and Services (MCS '12)*. 9–14.
- Aziz Mohaisen, Huy Tran, Abhishek Chandra, and Yongdae Kim. 2011. SocialCloud: Using Social Networks for Building Distributed Computing Services. *CoRR* abs/1112.2254 (2011).
- Nitinder Mohan and Jussi Kangasharju. 2016. Edge-Fog Cloud: A Distributed Cloud for Internet of Things Computations. In *IEEE 2nd Conference on Cloudification of Internet of Things (CIoT)*. 1–6.
- A. Moreno-Martínez, R. Gracia-Tinedo, M. Sánchez-Artigas, and P. García-López. 2012. FRIENDBOX: A cloudified F2F storage application. In *2012 IEEE 12th International Conference on Peer-to-Peer Computing (P2P)*. 75–76.
- Abderrahmen Mtibaa, Afnan Fahim, Khaled A. Harras, and Mostafa H. Ammar. 2013. Towards Resource Sharing in Mobile Device Clouds: Power Balancing Across Mobile Devices. *SIGCOMM Comput. Commun. Rev.* 43, 4 (Aug. 2013), 51–56.
- F. Nadeem, R. Prodan, and T. Fahringer. 2008a. Characterizing, Modeling and Predicting Dynamic Resource Availability in a Large Scale Multi-purpose Grid. In *2008 Eighth IEEE International Symposium on Cluster Computing and the Grid (CCGRID)*. 348–357.
- Farrukh Nadeem, Radu Prodan, Thomas Fahringer, and Vincent Keller. 2008b. Availability-based resource selection risk analysis in the grid. Technical report, CoreGRID Technical Report, Number TR-0169. (2008).
- R. Nakanishi. 2016. A Study on Utilization of TV sets in Volunteer Computing. *ITE Tech. Rep.* 40, 23 (July 2016), 17–20.
- Dirk Neumann, Christian Bodenstein, Omer F. Rana, and Ruby Krishnaswamy. 2011. STACEE: Enhancing Storage Clouds Using Edge Devices. In *Proceedings of the 1st ACM/IEEE Workshop on Autonomic Computing in Economics*. ACM, 19–26.
- S. H. Ngo, M. Fukushi, X. Jiang, and S. Horiguchi. 2008. Efficient Scheduling Schemes for Sabotage-Tolerance in Volunteer Computing Systems. In *22nd International Conference on Advanced Information Networking and Applications (aina 2008)*. 652–658.
- T. A. B. Nguyen, C. Meurisch, S. Niemczyk, C. Klos, D. Bohnstedt, and R. Steinmetz. 2017. Facilitating volunteer computing resources for in-network processing through message template. In *2017 International Conference on Networked Systems (NetSys)*. 1–2.
- Lei Ni, Aaron Harwood, and Peter J. Stuckey. 2006. Realizing the e-Science Desktop Peer Using a Peer-to-peer Distributed Virtual Machine Middleware. In *Proceedings of the 4th International Workshop on Middleware for Grid Computing (MCG '06)*. 3–8.
- S. A. Noor, R. Hasan, and M. M. Haque. 2014. CellCloud: A Novel Cost Effective Formation of Mobile Cloud Based on Bidding Incentives. In *7th IEEE International Conference on Cloud Computing*. 200–207.

- S. Al Noor, M. M. Hossain, and R. Hasan. 2016. SASCloud: Ad Hoc Cloud as Secure Storage. In *2016 IEEE International Conferences on Big Data and Cloud Computing (BDCloud), Social Computing and Networking (SocialCom), Sustainable Computing and Communications (SustainCom) (BDCloud-SocialCom-SustainCom)*. 37–44.
- Muhammad Nouman Durrani and Jawwad A. Shamsi. 2014. Review: Volunteer Computing: Requirements, Challenges, and Solutions. *J. Netw. Comput. Appl.* 39 (March 2014), 369–380.
- Daniel Nurmi, John Brevik, and Rich Wolski. 2005. Modeling Machine Availability in Enterprise and Wide-Area Distributed Computing Environments. In *Euro-Par 2005 Parallel Processing*, José C. Cunha and Pedro D. Medeiros (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 432–441.
- J. D. Osorio, H. Castro, and F. Brasileiro. 2012. Perspectives of UnaCloud: An Opportunistic Cloud Computing Solution for Facilitating Research. In *2012 12th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing*. 717–718.
- J. Panadero, L. Calvet, J. M. Marqués, and A. A. Juan. 2017. A simheuristic approach for resource allocation in volunteer computing. In *2017 Winter Simulation Conference (WSC)*. 1479–1490.
- H. Park and M. van der Schaar. 2010. Evolution of Resource Reciprocation Strategies in P2P Networks. *IEEE Transactions on Signal Processing* 58, 3 (March 2010), 1205–1218.
- K. B. Parmar, N. N. Jani, P. S. Shrivastav, and M. H. Patel. 2013. jUniGrid: A simplistic framework for integration of mobile devices in heterogeneous grid computing. *International Journal of Multidisciplinary Sciences and Engineering* 4, 1 (Jan 2013), 10–15.
- Kleiner Perkins Partners. 2017. Internet Trends 2017. (2017). <http://www.kpcb.com/internet-trends>
- E. N. Preeth, F. J. P. Mulerickal, B. Paul, and Y. Sastri. 2015. Evaluation of Docker containers based on hardware utilization. In *2015 International Conference on Control Communication Computing India (ICCC)*. 697–700.
- An Qin, Dian-Ming Hu, Jun Liu, Wen-Jun Yang, and Dai Tan. 2015. Fatman: Building Reliable Archival Storage Based on Low-Cost Volunteer Resources. *Journal of Computer Science and Technology* 30, 2 (01 Mar 2015), 273–282.
- R. Rahman, M. Meulpolder, D. Hales, J. Pouwelse, D. Epema, and H. Sips. 2010. Improving Efficiency and Fairness in P2P Systems with Effort-Based Incentives. In *2010 IEEE International Conference on Communications*. 1–5.
- K. Ramachandran, H. Lutfiyya, and M. Perry. 2010. Decentralized Resource Availability Prediction for a Desktop Grid. In *2010 10th IEEE/ACM International Conference on Cluster, Cloud and Grid Computing*. 643–648.
- Karthick Ramachandran, Hanan Lutfiyya, and Mark Perry. 2012. Decentralized Approach to Resource Availability Prediction Using Group Availability in a P2P Desktop Grid. *Future Gener. Comput. Syst.* 28, 6 (June 2012), 854–860.
- K. Ranganathan, M. Ripeanu, A. Sarin, and I. Foster. 2004. Incentive mechanisms for large collaborative resource sharing. In *IEEE International Symposium on Cluster Computing and the Grid, 2004. CCGrid 2004*. 1–8.
- Xiaojuan Ren, Seyong Lee, Rudolf Eigenmann, and Saurabh Bagchi. 2007. Prediction of Resource Availability in Fine-Grained Cycle Sharing Systems Empirical Evaluation. *Journal of Grid Computing* 5, 2 (01 Jun 2007), 173–195.
- A. Rezgui, N. Davis, Z. Malik, B. Medjahed, and H. Soliman. 2018. CloudFinder: A System for Processing Big Data Workloads on Volunteered Federated Clouds. *IEEE Transactions on Big Data* (2018), 1–1.
- B. Rood and M. J. Lewis. 2008. Resource Availability Prediction for Improved Grid Scheduling. In *2008 IEEE Fourth International Conference on eScience*. 711–718.
- M. Ryden, K. Oh, A. Chandra, and J. Weissman. 2014. Nebula: Distributed edge cloud for data-intensive computing. In *2014 International Conference on Collaboration Technologies and Systems (CTS)*. 491–492.
- Jordi Sabater and Carles Sierra. 2002. Reputation and Social Network Analysis in Multi-agent Systems. In *Proceedings of the First International Joint Conference on Autonomous Agents and Multiagent Systems: Part 1 (AAMAS '02)*. 475–482.
- L. F. G. Sarmenta. 2001. Sabotage-tolerance mechanisms for volunteer computing systems. In *Proceedings First IEEE/ACM International Symposium on Cluster Computing and the Grid*. 337–346.
- R. Schmohl and U. Baumgarten. 2008. Heterogeneity in mobile computing environments. In *Proceedings of the 2008 international conference on wireless networks*.
- Sven Schulz, Wolfgang Blochinger, Markus Held, and Clemens Dangelmayr. 2008. COHESION A microkernel based Desktop Grid platform for irregular task-parallel applications. *Future Generation Computer Systems* 24, 5 (2008), 354 – 370.
- Stefan Seifert. 2006. *Posted Price Offers in Internet Auction Markets*. Springer, Berlin, Heidelberg.
- A. Shahri, M. Hosseini, R. Ali, and F. Dalpiaz. 2014. Gamification for Volunteer Cloud Computing. In *2014 IEEE/ACM 7th International Conference on Utility and Cloud Computing*. 616–617.
- Cong Shi, Vasileios Lakafosis, Mostafa H. Ammar, and Ellen W. Zegura. 2012. Serendipity: Enabling Remote Computing Among Intermittently Connected Mobile Devices. In *Proceedings of the Thirteenth ACM International Symposium on Mobile Ad Hoc Networking and Computing (MobiHoc '12)*. ACM, New York, NY, USA, 145–154.
- W. Shi and S. Dustdar. 2016. The Promise of Edge Computing. *Computer* 49, 5 (May 2016), 78–81.
- K. Shudo, Y. Tanaka, and S. Sekiguchi. 2005. P3: P2P-based middleware enabling transfer and aggregation of computational resources. In *CCGrid 2005. IEEE International Symposium on Cluster Computing and the Grid, 2005.*, Vol. 1. 259–266.
- Major Singh and Lakhwinder Kaur. 2013. Resource Grouping in Grid Environment Towards the Availability and Reliability of Computing Service. *Journal of Advanced Computing* 1, 1 (2013), 1–8.

- Alicia M Sintes. 2006. Recent results on the search for continuous sources with LIGO and GEO 600. *Journal of Physics: Conference Series* 39, 1 (2006), 36. <http://stacks.iop.org/1742-6596/39/i=1/a=008>
- Michael Sirivianos, Jong Han Park, Rex Chen, and Xiaowei Yang. 2007. Free-riding in BitTorrent Networks with the Large View Exploit. In *IPTPS*.
- Y. E. Soelistio. 2013. Application distribution model in volunteer computing environment using peer-to-peer torrent like approach. In *2013 International Conference on Computer, Control, Informatics and Its Applications (IC3INA)*. 261–266.
- J. Sonnek, A. Chandra, and J. Weissman. 2007. Adaptive Reputation-Based Scheduling on Unreliable Distributed Infrastructures. *IEEE Transactions on Parallel and Distributed Systems* 18, 11 (Nov 2007), 1551–1564.
- David Stainforth, Jamie Kettleborough, Andrew Martin, Andrew Simpson, Richard Gillis, Ali Akkas, Richard Gault, Mat Collins, David Gavaghan, and Myles Allen. 2002. Climateprediction.net: Design principles for public-resource modeling research. In *14th IASTED International Conference Parallel and Distributed Computing and Systems*. 32–38.
- Cristiano Tapparello, Colin Funai, Shuroq Hijazi, Abner Aquino, Bora Karaoglu, He Ba, Jiye Shi, and Wendi Heinzelman. 2015. *Volunteer Computing on Mobile Devices: State of the Art and Future Research Directions*. IGI Global, Hershey, PA, 153–181.
- M. Taufer, A. Kerstens, T. P. Estrada, D. A. Flores, R. Zamudio, P. J. Teller, R. Armen, and C. L. Brooks. 2007. Moving Volunteer Computing towards Knowledge-Constructed, Dynamically-Adaptive Modeling and Scheduling. In *2007 IEEE International Parallel and Distributed Processing Symposium*. 1–8.
- DM Toth. 2008. Improving the productivity of volunteer computing. Worcester Polytechnic Institute. (2008).
- D. Toth and D. Finkel. 2008. Increasing the amount of work completed by volunteer computing projects with task distribution policies. In *EEE International Symposium on Parallel and Distributed Processing*. 1–8.
- David Toth and David Finkel. 2009. Improving the Productivity of Volunteer Computing by Using the Most Effective Task Retrieval Policies. *Journal of Grid Computing* 7, 4 (28 Aug 2009), 519.
- Etienne Urbah, Peter Kacsuk, Zoltan Farkas, Gilles Fedak, Gabor Kecskemeti, Oleg Lodygensky, Attila Marosi, Zoltan Balaton, Gabriel Caillat, Gabor Gombas, Adam Kornafeld, Jozsef Kovacs, Haiwu He, and Robert Lovas. 2009. EDGeS: Bridging EGEE to BOINC and XtremWeb. *Journal of Grid Computing* 7, 3 (16 Sep 2009), 335–354.
- Blesson Varghese and Rajkumar Buyya. 2018. Next generation cloud computing: New trends and research directions. *Future Generation Computer Systems* 79 (2018), 849 – 861.
- S. S. Vazhkudai, Xiaosong Ma, V. W. Freeh, J. W. Strickland, N. Tammineedi, and S. L. Scott. 2005. FreeLoader: Scavenging Desktop Storage Resources for Scientific Data. In *Supercomputing, 2005. Proceedings of the ACM/IEEE SC 2005 Conference*. 56–56.
- D. Vega, R. Meseguer, F. Freitag, and S. F. Ochoa. 2013. Effort-based incentives for resource sharing in collaborative volunteer applications. In *Proceedings of the 2013 IEEE 17th International Conference on Computer Supported Cooperative Work in Design (CSCWD)*. 37–42.
- C. A. Waldspurger, T. Hogg, B. A. Huberman, J. O. Kephart, and W. S. Stornetta. 1992. Spawn: a distributed computational economy. *IEEE Transactions on Software Engineering* 18, 2 (Feb 1992), 103–117.
- Xiaofeng Wang, Chee Shin Yeo, Rajkumar Buyya, and Jinshu Su. 2011. Optimizing the makespan and reliability for workflow applications with reputation and a look-ahead genetic algorithm. *Future Generation Computer Systems* 27, 8 (2011), 1124 – 1134.
- K. Watanabe and M. Fukushi. 2010. Generalized Spot-Checking for Sabotage-Tolerance in Volunteer Computing Systems. In *2010 10th IEEE/ACM International Conference on Cluster, Cloud and Grid Computing*. 655–660.
- Kan Watanabe, Masaru Fukushi, and Michitaka Kameyama. 2011. Adaptive Group-Based Job Scheduling for High Performance and Reliable Volunteer Computing. *Journal of Information Processing* 19 (2011), 39–51.
- Jon B. Weissman, Pradeep Sundararajan, Abhishek Gupta, Matthew Ryden, Rohit Nair, and Abhishek Chandra. 2011. Early Experience with the Distributed Nebula Cloud. In *Proceedings of the 4th International Workshop on Data-intensive Distributed Computing*. 17–26.
- E. Williams. 2004. Energy intensity of computer manufacturing: hybrid assessment combining process and economic input/output methods. *Environmental Science & Technology* 38, 22 (2004), 61661–6174.
- P. Wu and G. Wu. 2009. Reputation Mechanism in Peer-to-Peer Network. In *2009 First International Conference on Information Science and Engineering*. 1793–1796.
- Jun Zhang and Chris Phillips. 2011. Job Scheduling via Resource Availability Prediction for Volunteer Computational Grids. *Int. J. Grid Util. Comput.* 2, 1 (May 2011), 25–32.
- Z. Zheng, S. Xie, H. Dai, X. Chen, and H. Wang. 2017. An Overview of Blockchain Technology: Architecture, Consensus, and Future Trends. In *2017 IEEE International Congress on Big Data (BigData Congress)*. 557–564.
- D. Zhou and V. L. 2004. Cluster Computing on the Fly: resource discovery in a cycle sharing peer-to-peer system. In *IEEE International Symposium on Cluster Computing and the Grid, 2004. CCGrid 2004*. 66–73.
- H. Zhoujun, H. Zhigang, and L. Zhenhua. 2007. Resource Availability Evaluation in Service Grid Environment. In *The 2nd IEEE Asia-Pacific Service Computing Conference (APSCC 2007)*. 232–238.